Secure Computation - Examples

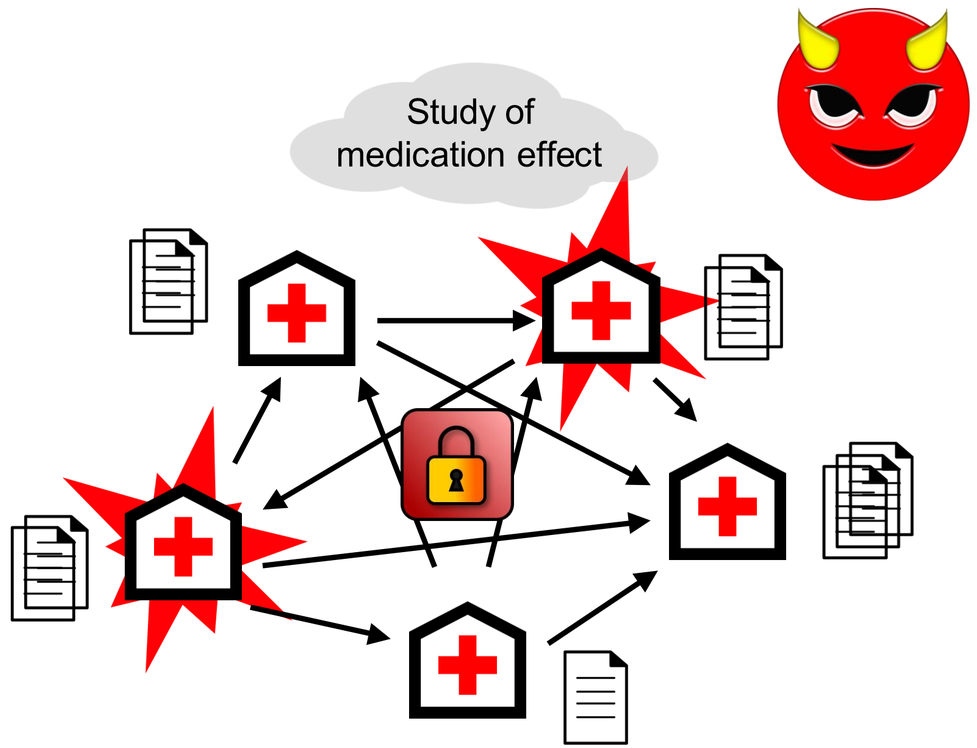
Scenarios

Hospitals hold private data of cancer patients. Access to this data would benefit to cancer research (statistics, machine learning to develop treatments, etc). The data can legally and morally not be shared.

Is it possible to compute statistics on the joint data held by hospitals, without seeing the data?

We want two properties:

- **Correctness:** everyone learns the result of the computation
- **Privacy:** nothing more than the result is learned



Model (more on that later):

- Point-to-point secure, authenticated network
- Polytime, probabilistic, interactive algorithms
- An adversary can corrupt (control) a subset of the parties

Credit: slides adapted starting from the ones of Geoffrey Couteau

Secure Computation - Examples

More Scenarios

Auctions

E-voting

Secure computation

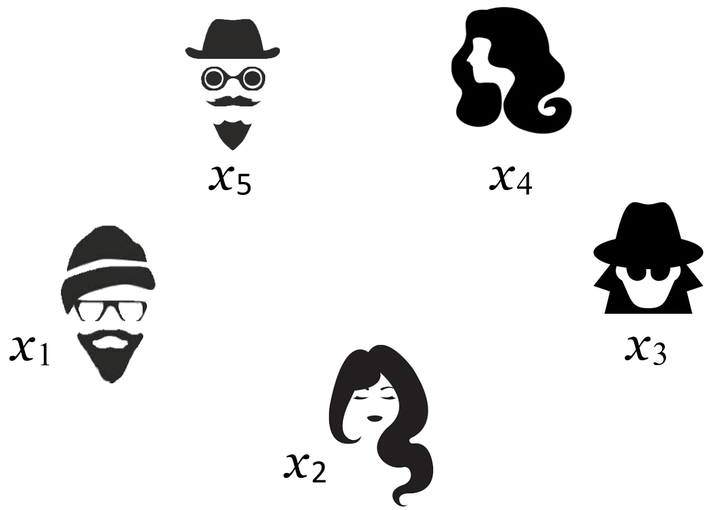
Tax-fraud detection

Contact tracing

Studies

The Model

Real World

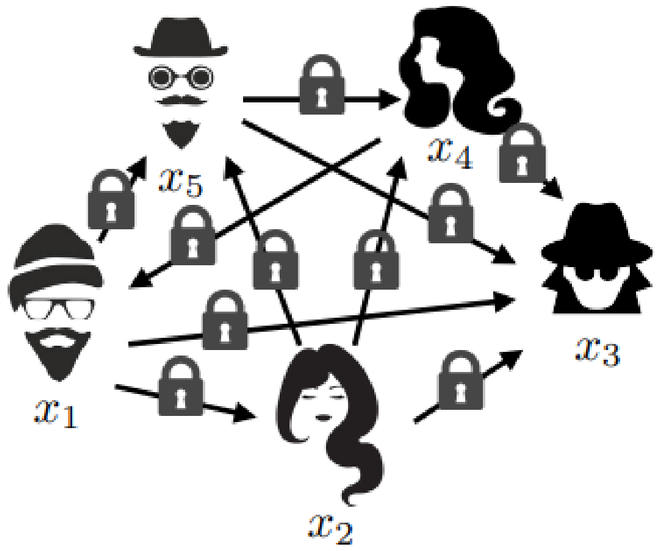


Goal

- Public function f
- All players want to get $f(x_1, x_2, x_3, x_4, x_5)$
- No player should learn anything more

The Model

Real World



Network model

- Fully authenticated network (with signatures)
- Two communication models

1. The broadcast (or blackboard) network

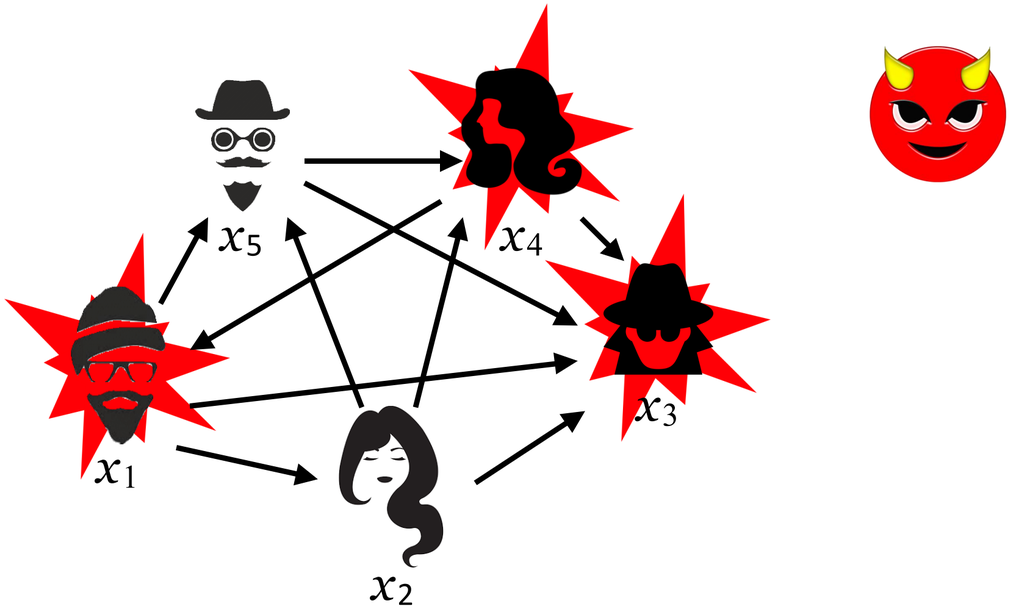
   Each party with message **m** can write it on a public blackboard. Everyone can see what is written on the board. All messages are authenticated.

2. The point-to-point network

   All parties are connected through a complete point-to-point network. Each channel is perfectly authenticated *and* private.

The Model

Real World



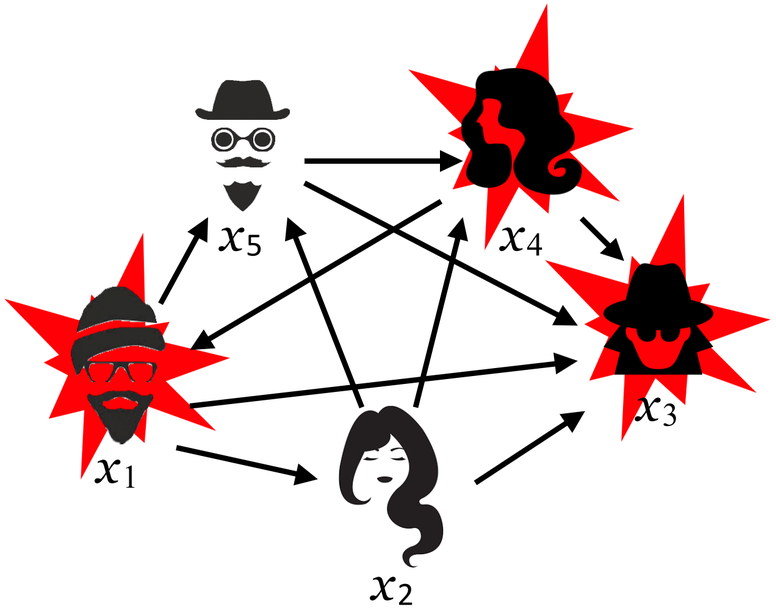
Adversarial model

- An adversary can corrupt a subset of the players

The adversary sets the input and randomness of corrupted player and reads their output, that can include the exchanged messages

The Model

Real World



Adversarial model

- An adversary can corrupt a subset of the players
- Two standard corruption models

1. Honest-but-curious corruption

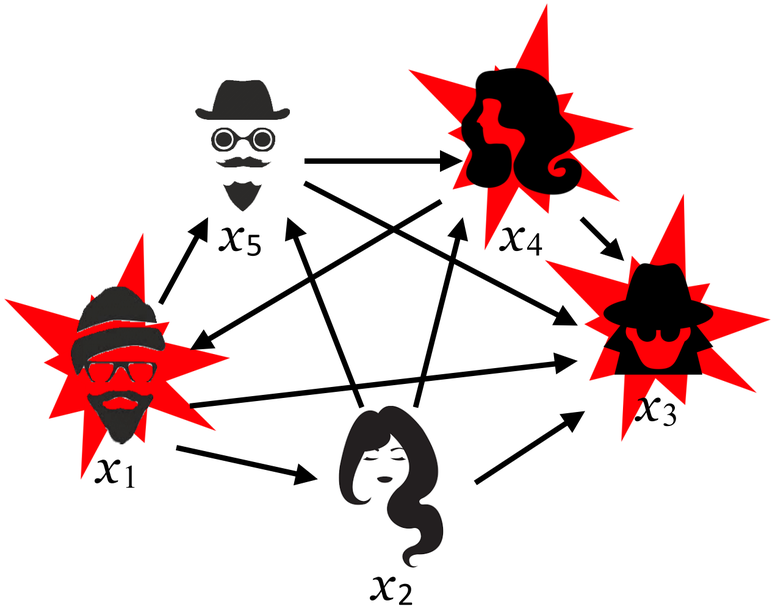
   The corrupted parties follow the specification of the protocol. The adversary is passive: he tries to retrieve private information by observing the transcript (e.g., he does not want to be caught, or he is honest, saves data, but later on get corrupted).

2. Malicious corruption

   The adversary fully control the corrupted parties, and can make them behave arbitrarily in the protocol.

The Model

Real World



Adversarial model

- An adversary can corrupt a subset of the players
- Two standard corruption models
- Two standard corruption levels

1. Honest majority

   The adversary can simultaneously corrupt only a strict minority of the players.

2. Dishonest majority

   The adversary can corrupt all-but-one players.

From "Introduction to Modern Cryptography", by Katz and Lindell, highly recommended to give a look at basic notions and constructions.

DEFINITION 8.29 *Two probability ensembles* $\mathcal{X} = \{X_n\}_{n\in\mathbb{N}}$ *and* $\mathcal{Y} = \{Y_n\}_{n\in\mathbb{N}}$ *are* computationally indistinguishable, *denoted* $\mathcal{X} \overset{c}{\equiv} \mathcal{Y}$, *if for every probabilistic polynomial-time distinguisher* D *there exists a negligible function* negl *such that:*

$$\left| \Pr_{x \leftarrow X_n}[D(1^n, x) = 1] - \Pr_{y \leftarrow Y_n}[D(1^n, y) = 1] \right| \leq \mathsf{negl}(n).$$

The Model - Defining Security

Computational Indistinguishability - Example

$$(\mathbb{G}, g, p) \leftarrow \mathsf{GroupGen}(1^\lambda)$$

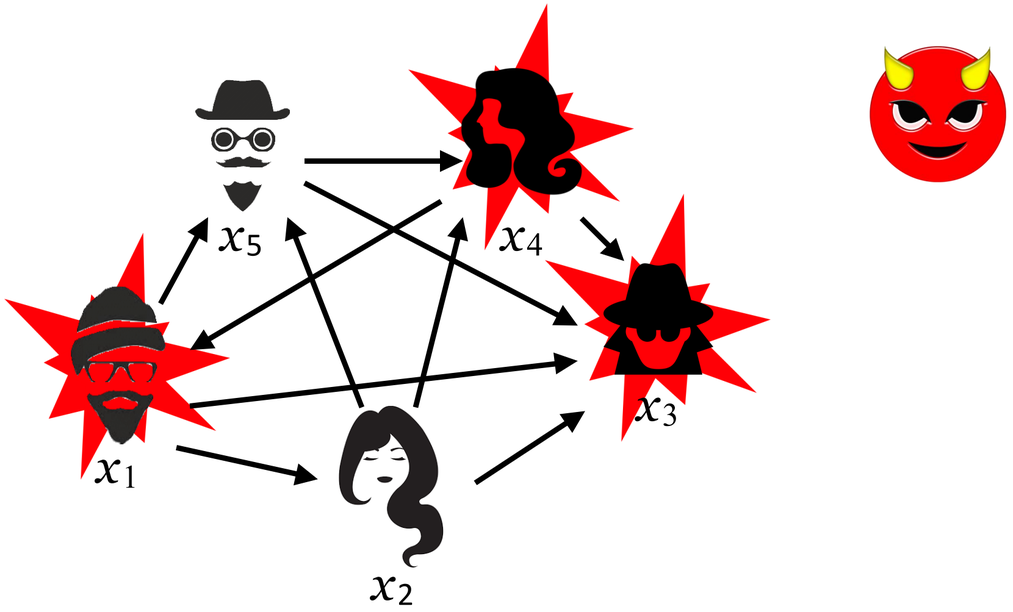
$$\boxed{\{(g^a, g^b, g^{ab}) \mid (a,b) \leftarrow \mathbb{Z}_p^2\} \approx \{(g^a, g^b, g^c) \mid (a,b,c) \leftarrow \mathbb{Z}_p^3\}}$$

(this is the Decisional Diffie-Hellman assumption)

We will sometimes consider also statistical and perfect indistinguishability. These 3 concepts are fundamental and thus make sure you understand them. Refer to basic crypto books or suggested readings for the full definitions and to get familiar with their meaning.

The Model - Defining Security

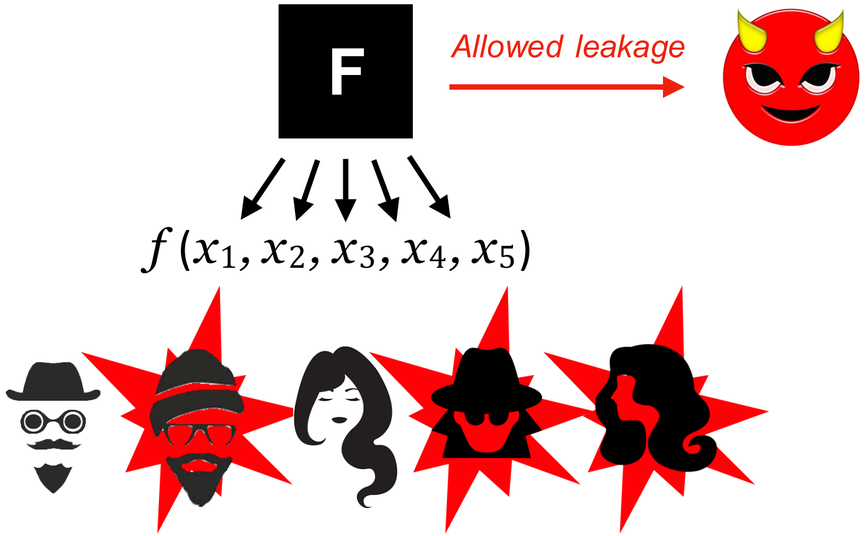
Real World



Real Behavior

- Players interact through [network model]
- Some players are corrupted in [corruption model]

Ideal World

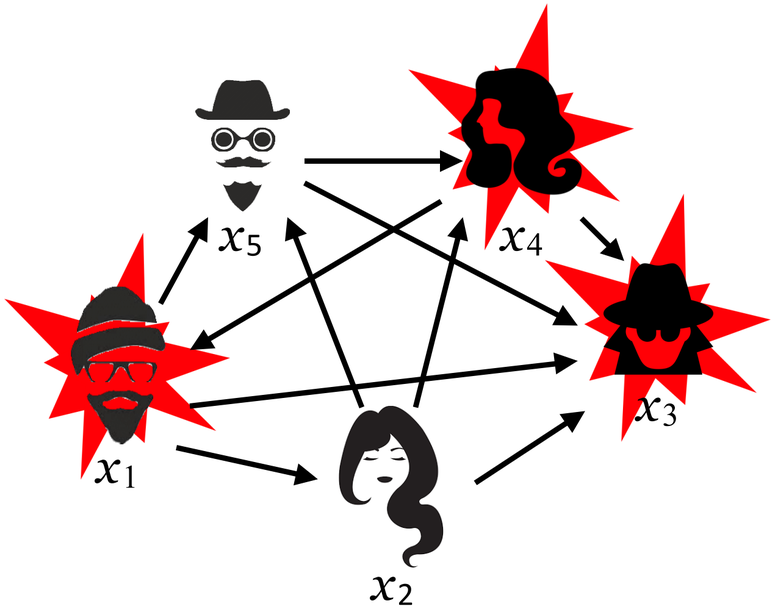


Ideal Behavior

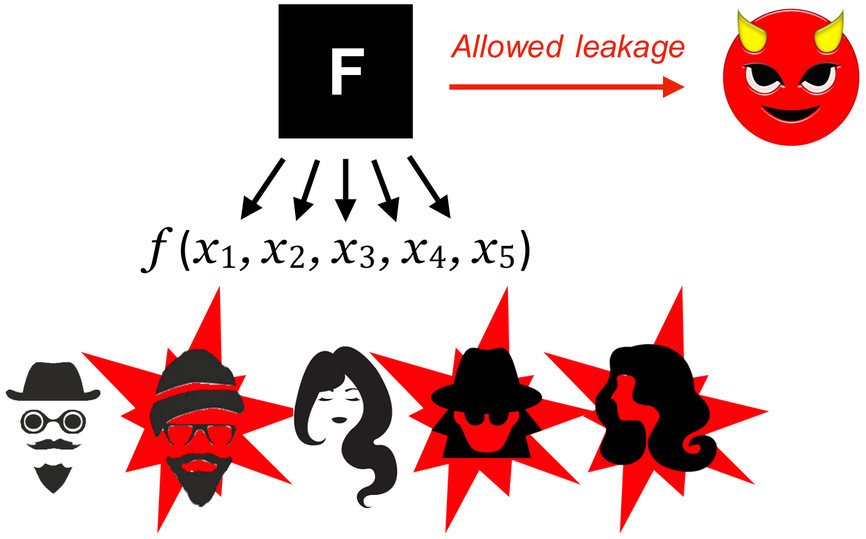
- Parties send their inputs to a trusted party **F** through perfectly secure authenticated channels
- **F** computes the output and reveals the result to each party
- The adversary only gets some allowed leakage (+ input/output of corrupted parties)

The Model - Defining Security

Real World



Ideal World



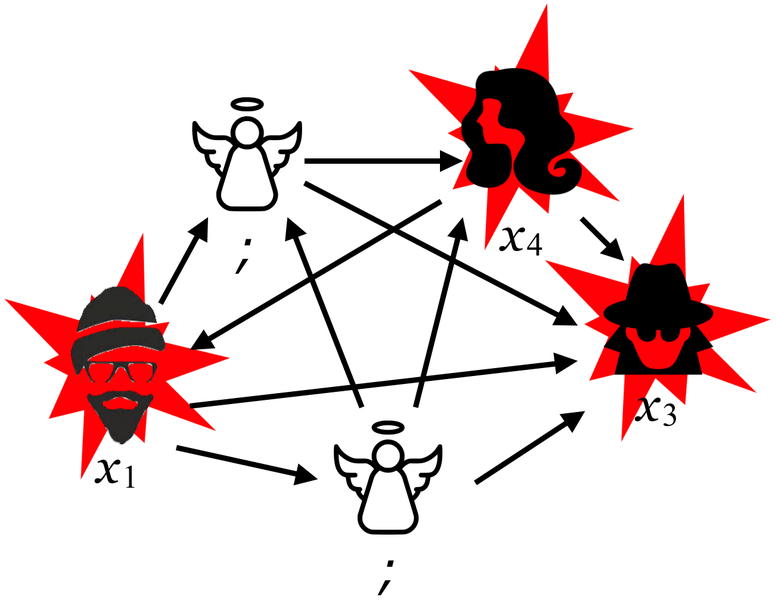
Simulation

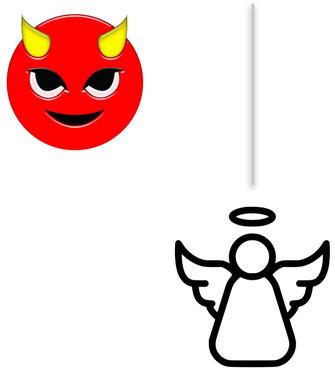
Core idea: we construct a *simulator* which fools the adversary into believing he is playing the real-world protocol, while making him effectively play the ideal world protocol. Then, we prove that *no adversary* can distinguish the simulated protocol from the real protocol.

Take the time to convince yourself that this guarantees that the real protocol is as secure as the ideal functionality. If you cannot convince yourself, please ask.

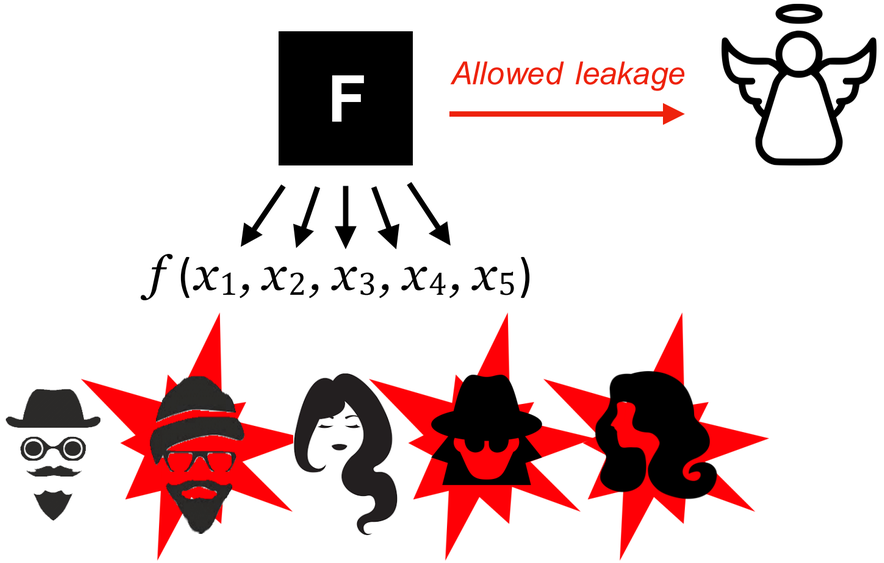
The Model - Defining Security

Real World





Ideal World

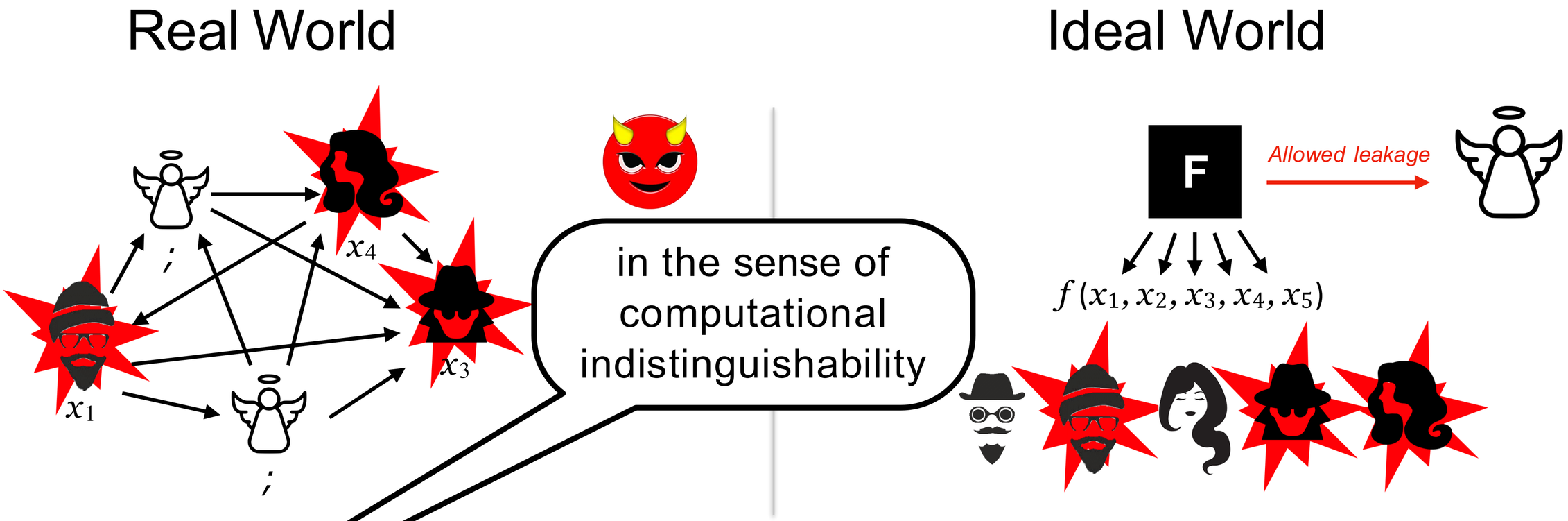


Simulation

Core idea: we construct a *simulator* which fools the adversary into believing he is playing the real world protocol, while making him effectively play the ideal world protocol. Then, we prove that *no adversary* can distinguish the simulated protocol from the real protocol.

emulates the *honest parties* in the real world, and the *adversary* in the ideal world.

The Model - Defining Security



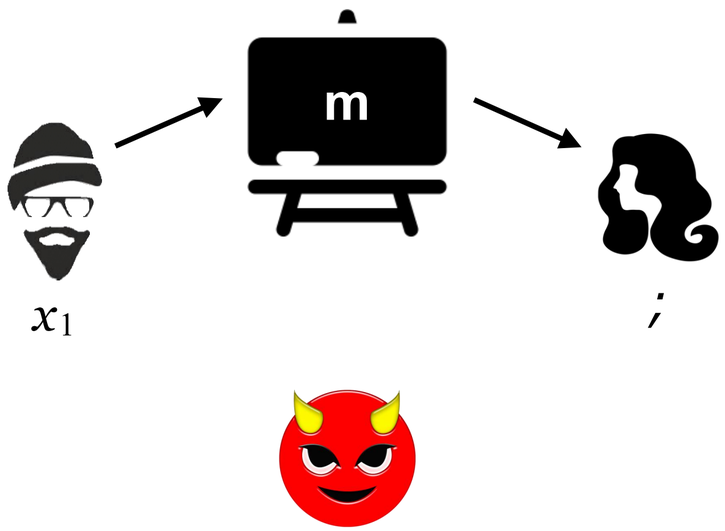
Simulation

Core idea: we construct a *simulator* which fools the adversary into believing he is playing the real world protocol, while making him effectively play the ideal world protocol. Then, we prove that *no adversary* can distinguish the simulated protocol from the real protocol.

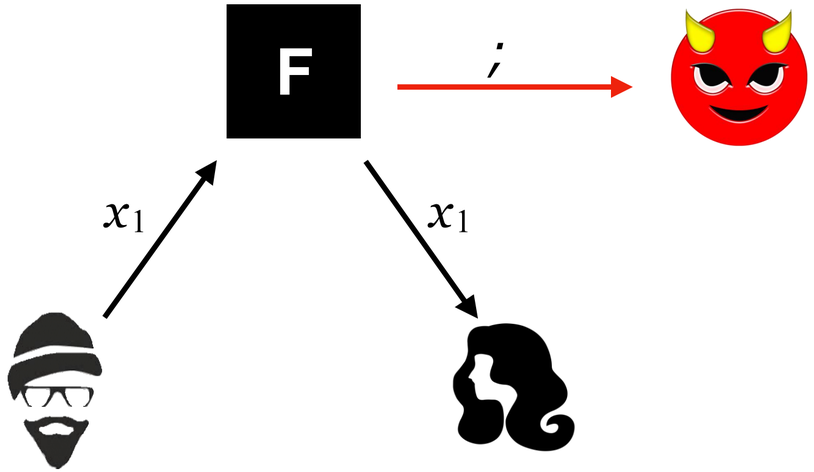
The **distributions** of the adversary's **view + (guess why) the input/output of honest parties** in the real world and in the simulated world are **computational (or even statistical/perfect) indistinguishable**

Simple Example: Secure Communication

Real World



Ideal World



Any idea how to do that?

Real Behavior

- Blackboard model (public communication)
- No corruption (but the adversary sees the board)

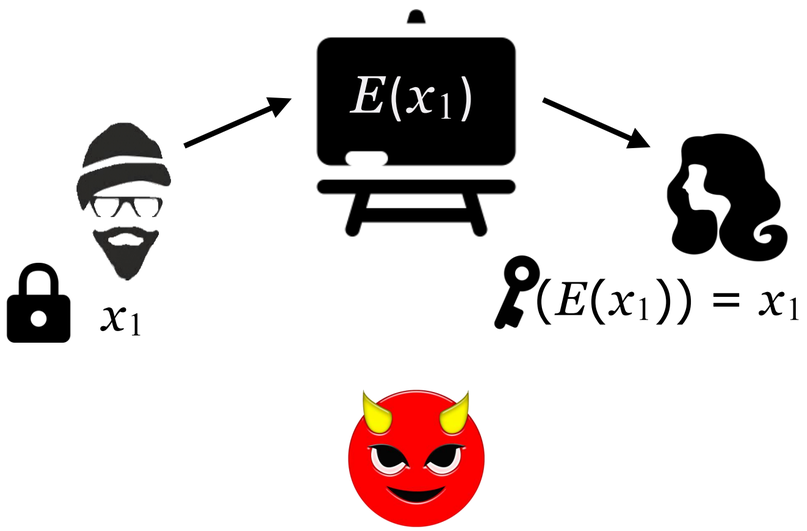
Ideal Behavior

- The sender sends his input to **F**
- **F** sends it to the receiver; no leakage.

";" denotes the empty string

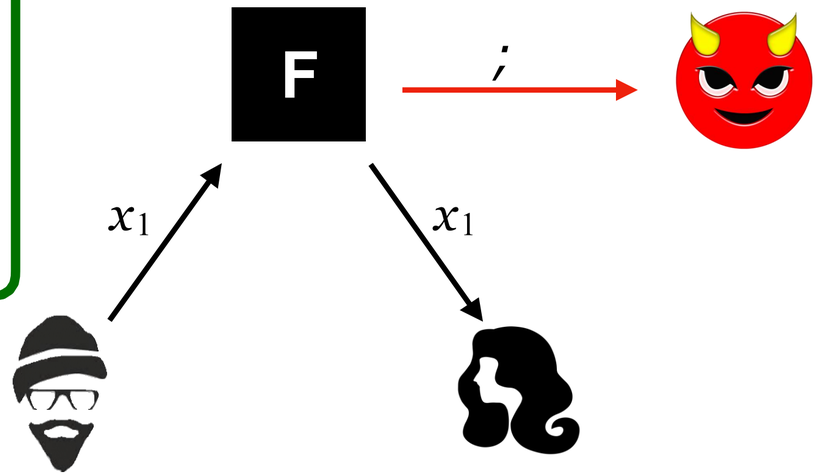
Simple Example: Secure Communication

Real World



E = IND-CPA secure encryption scheme. Sender message: input encrypted with the public key of the receiver.

Ideal World



Real Behavior

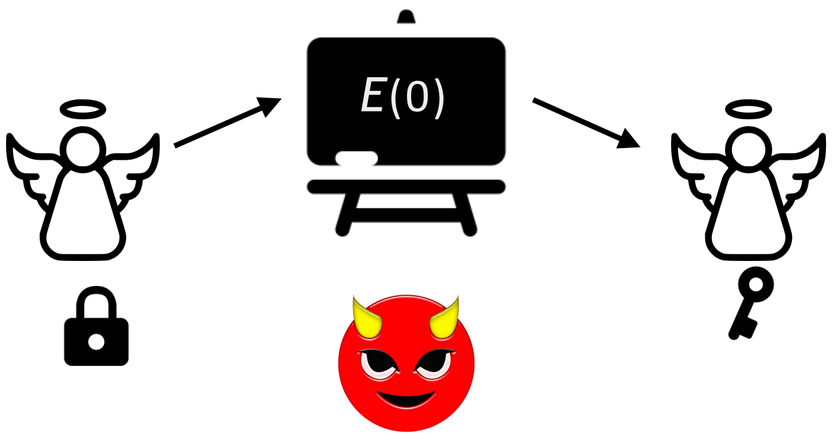
- Blackboard model (public communication)
- No corruption (but the adversary sees the board)

Ideal Behavior

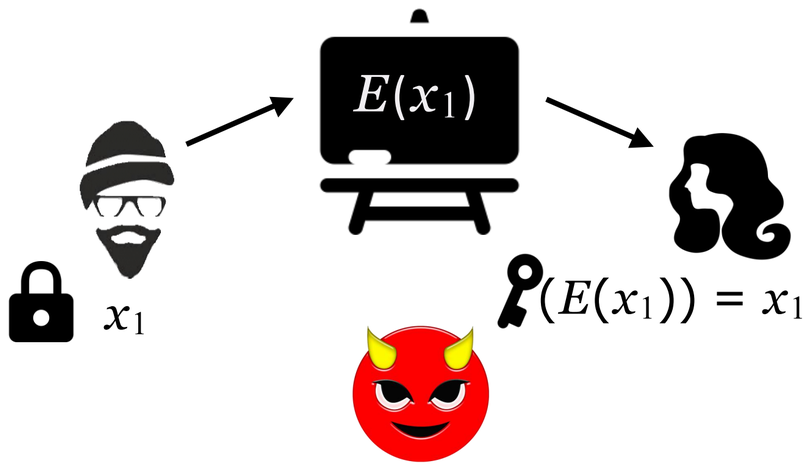
- The sender sends his input to **F**
- **F** sends it to the receiver; no leakage.

Simple Example: Secure Communication

Simulation



Reality



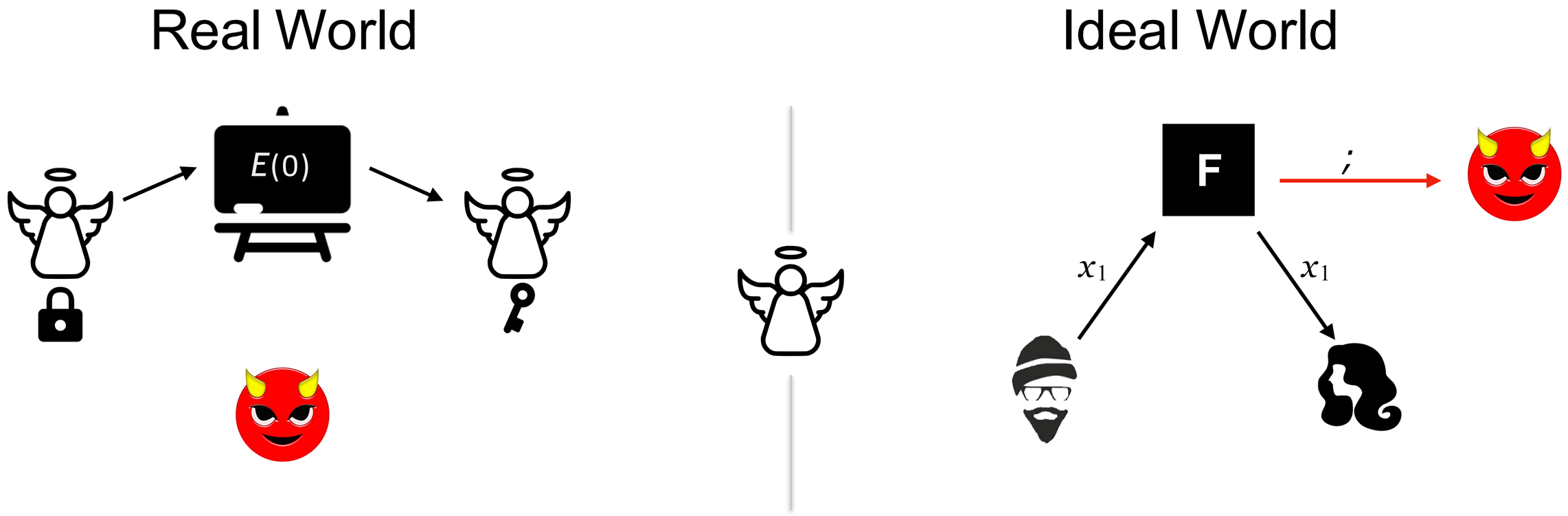
Simulation

Simulating is easy: generates a public key honestly.

Simulating is harder, since does not know x_1.

Solution: encrypt an arbitrary value instead!

Simple Example: Secure Communication



The simulation is indistinguishable from the real protocol if E is IND-CPA secure, hence the protocol securely *emulates* the ideal functionality F under the assumption that E is IND-CPA secure.

Simple Example: Secure Communication

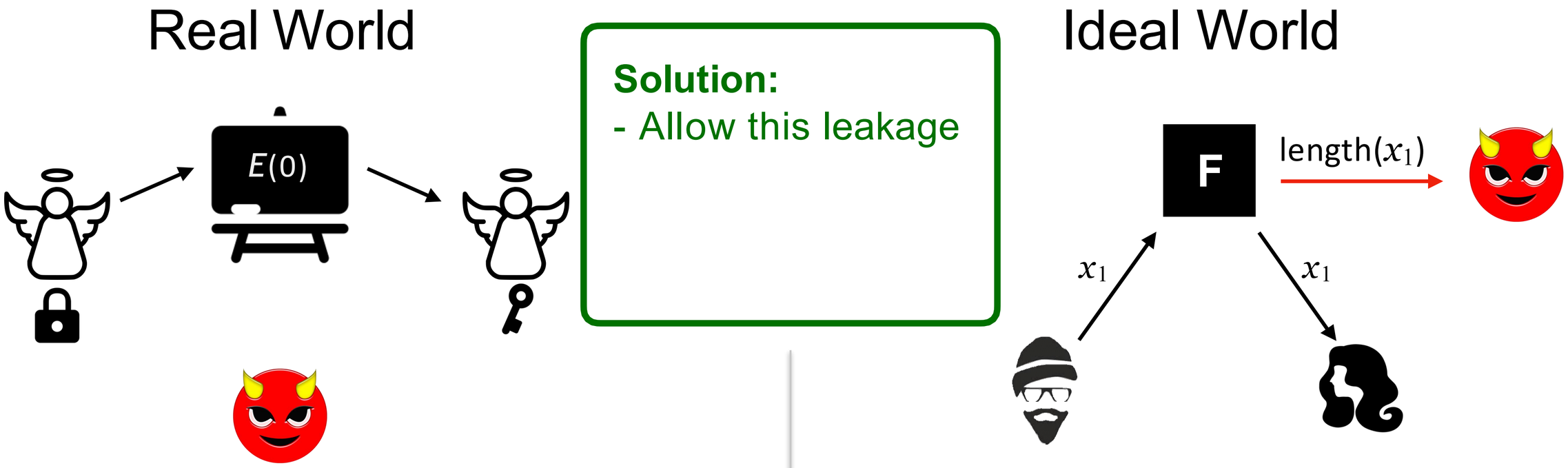
Simulation

View: 🔒 , $E(0)$

View: 🔒 , $E(x_1)$

Cannot work: the plaintexts do not have the same size!

Simple Example: Secure Communication



Solution:
- Allow this leakage

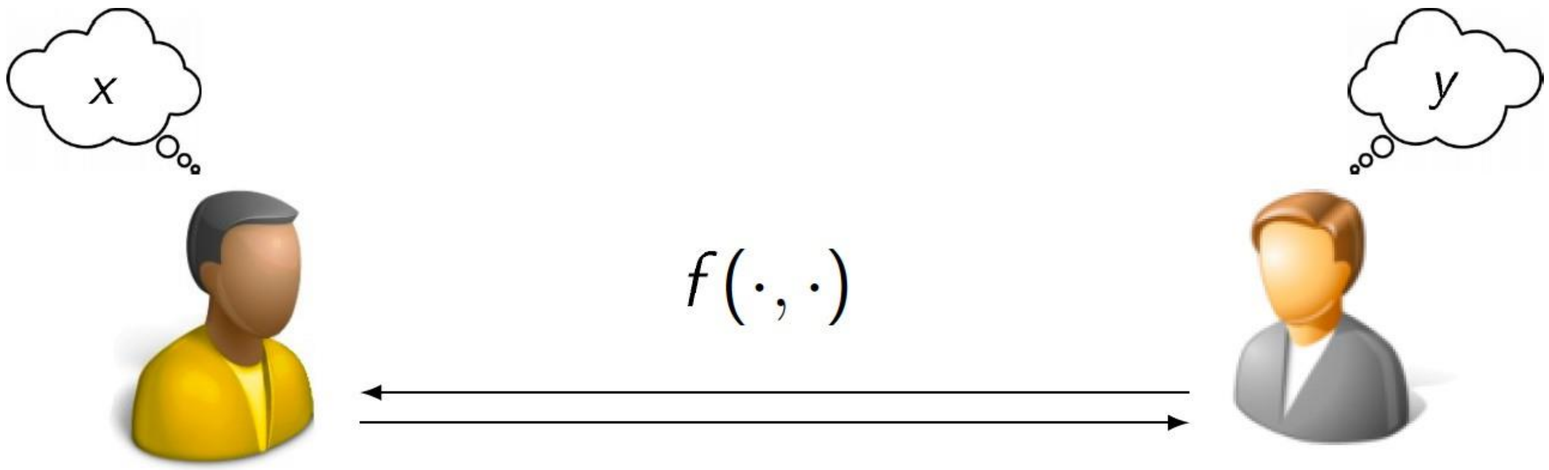
Simulation

View: 🔒 , $E(0)$

View: 🔒 , $E(x_1)$

⚠ **Cannot work: the plaintexts do not have the same size!**

Two-Party Secure Computation for *All* Functions



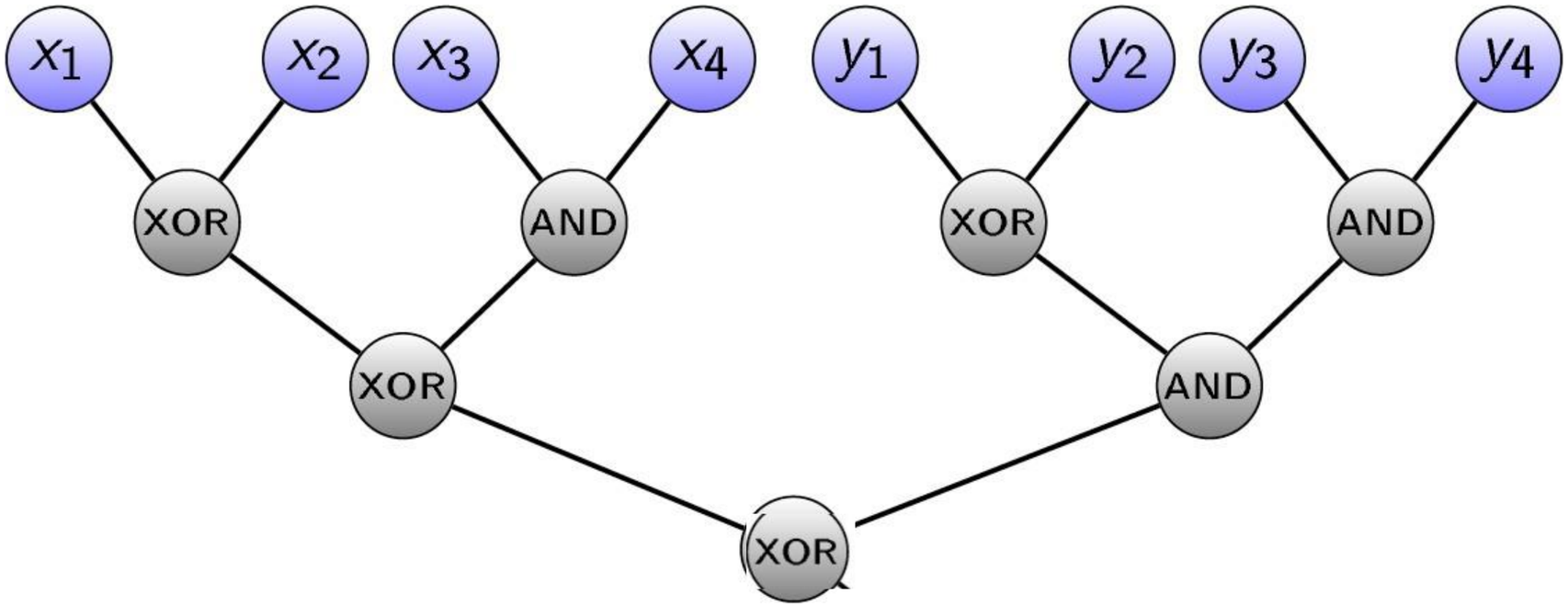
Goal: $f(x, y)$

Idea: represent f as a boolean circuit

Building Block I: Boolean Circuits

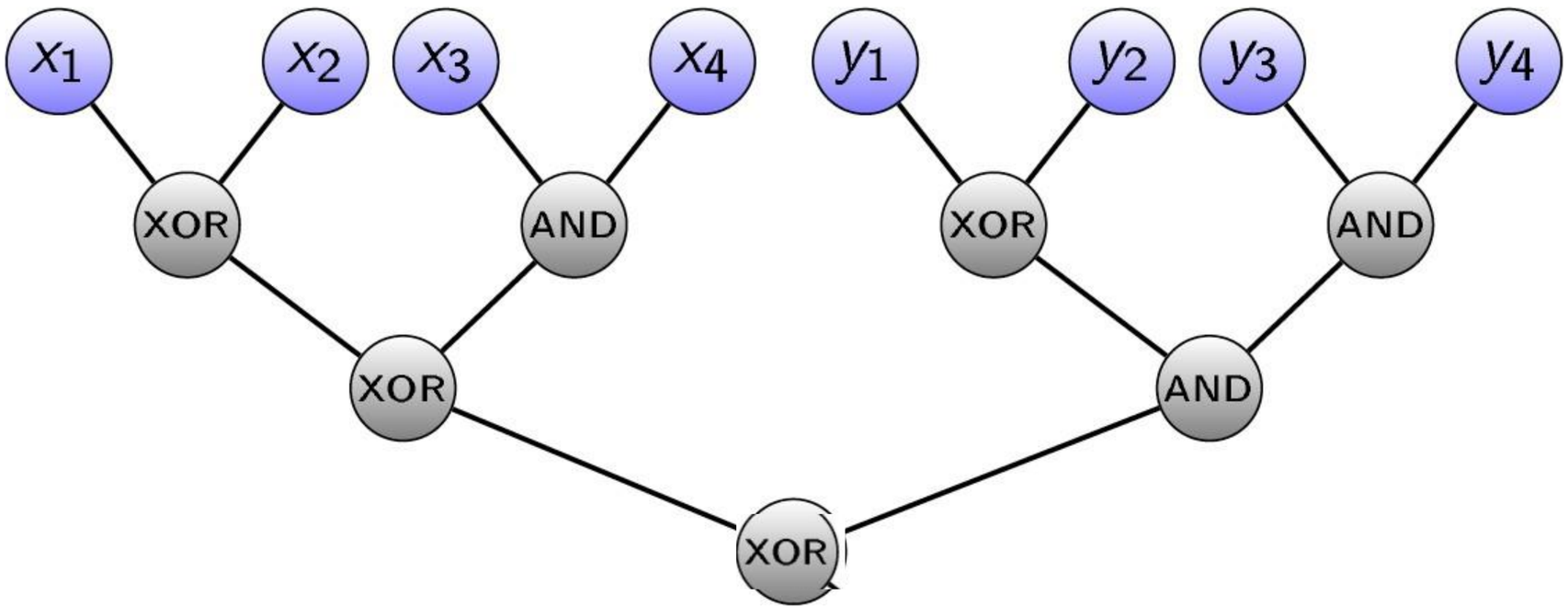
Useful fact:

any polytime-computable function can be computed by a poly-size boolean circuit over the {XOR, AND} bases.



Building Block I: Boolean Circuits

Idea: « encrypting » the gates such that they can only be evaluated given appropriate keys, and while hiding their exact behavior.



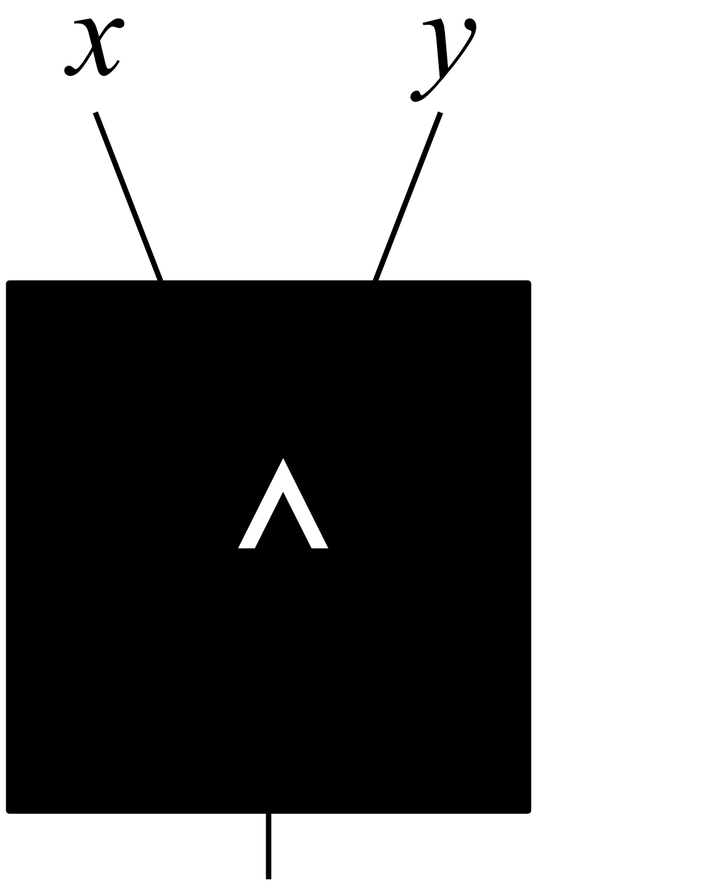
Building Block II: Symmetric Encryption

We let $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ be a symmetric encryption scheme with the following properties:

- **KeyGen generates a key**
- **$\mathsf{Enc}_K(m) \rightarrow c$ generates a random encryption of the plaintext**
- **$\mathsf{Dec}_K(c)$ returns if $c = \mathsf{Enc}_K(m)$**
- A decryption of a ciphertext with a *wrong* key K' returns « error » (hence, it reveals that a wrong key was used)

Building Block II: Symmetric Encryption

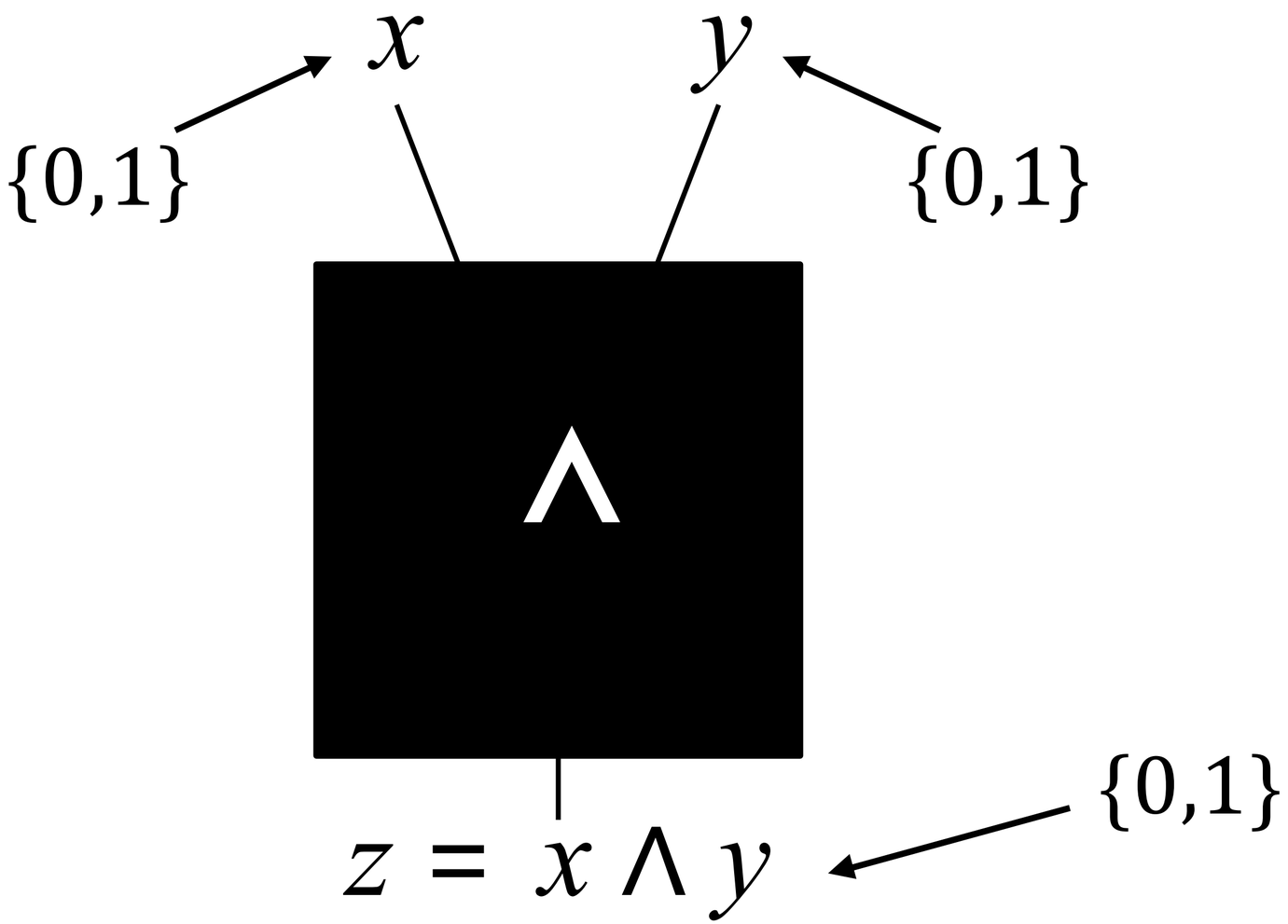
We will use this encryption scheme to « encrypt » logical gates.



$$z = x \wedge y$$

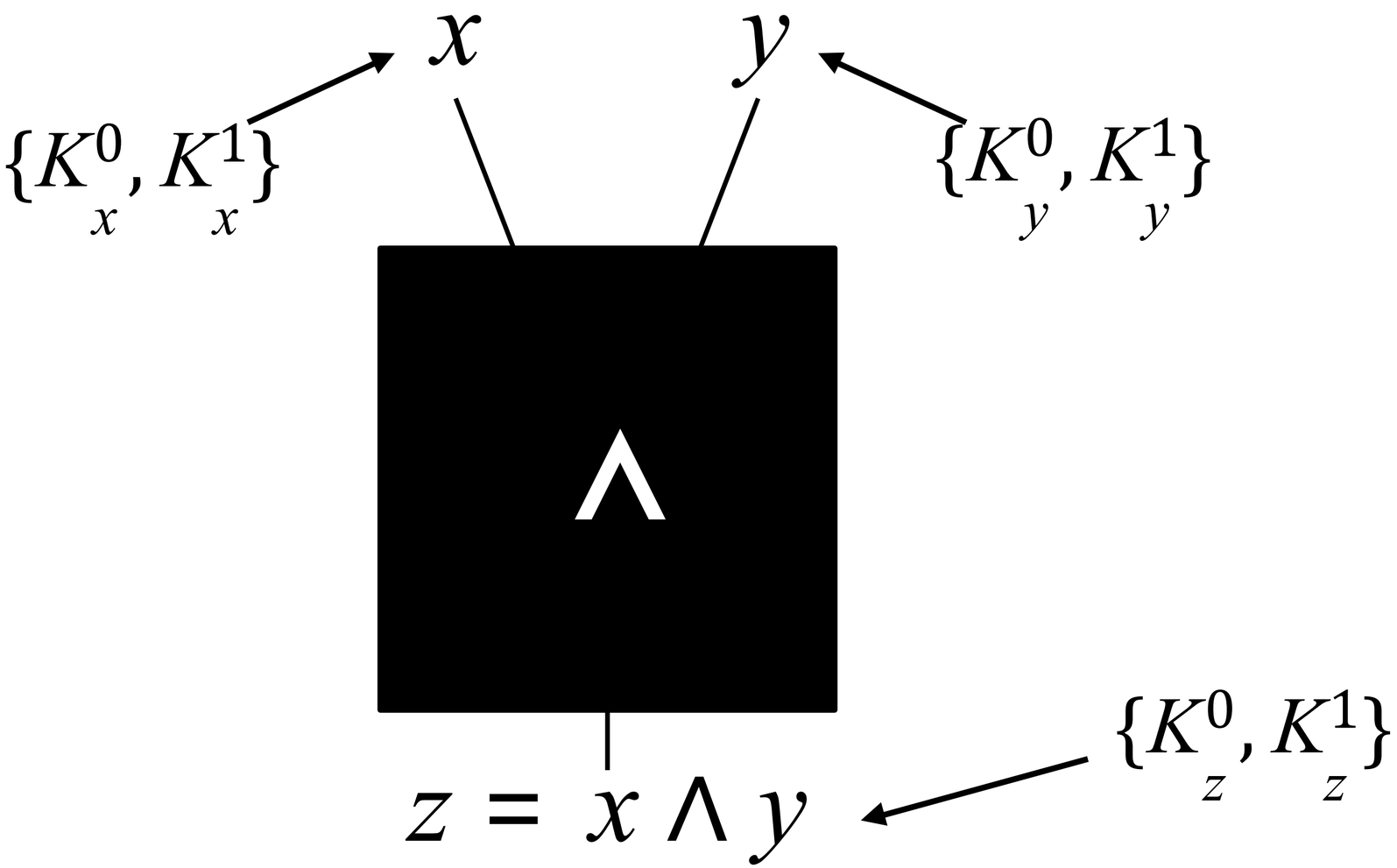
Building Block II: Symmetric Encryption

The inputs and outputs are bits, but we will « represent » them using random keys, to hide their true value:



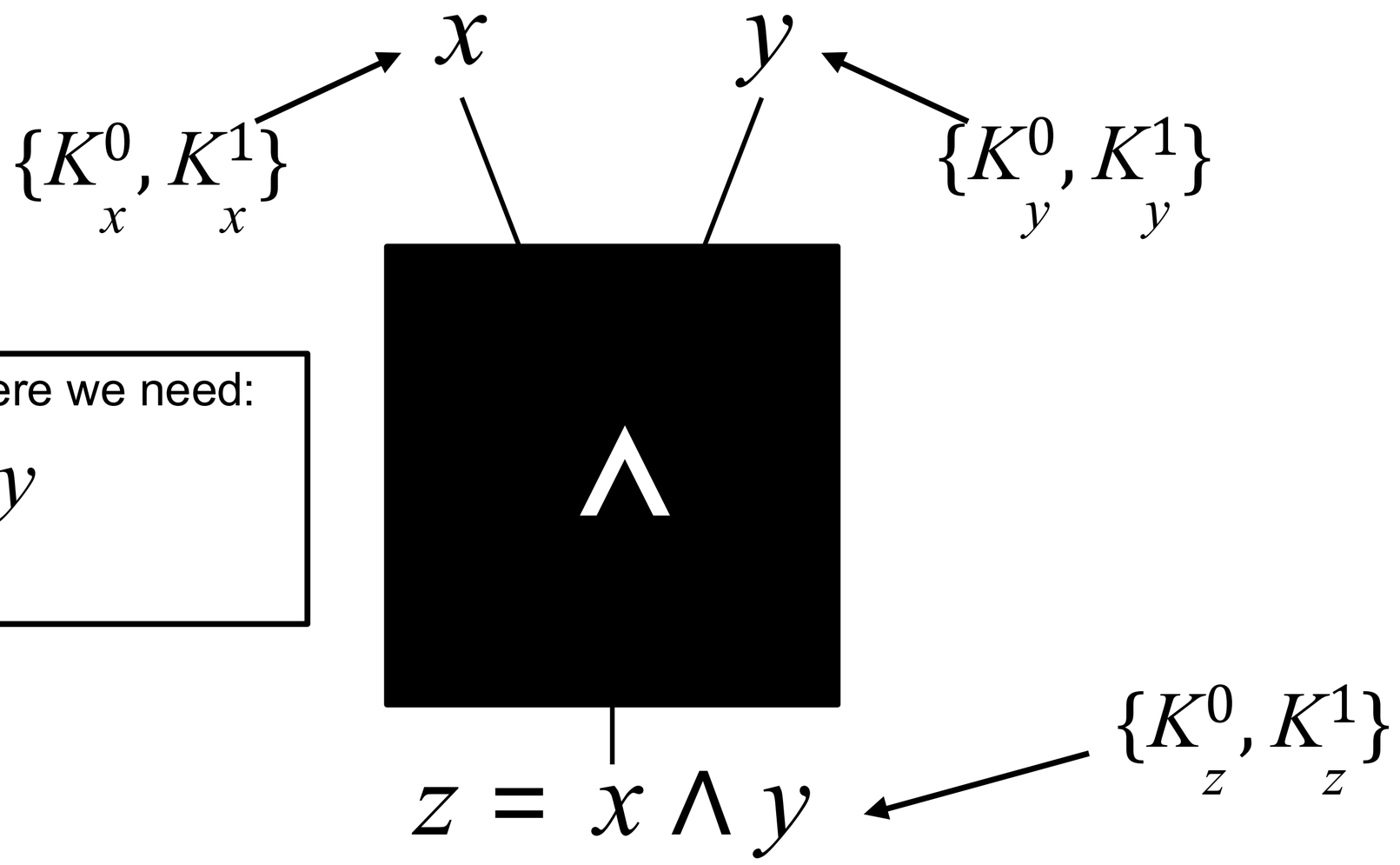
Building Block II: Symmetric Encryption

The inputs and outputs are bits, but we will « represent » them using random keys, to hide their true value:



Building Block II: Symmetric Encryption

The inputs and outputs are bits, but we will « represent » them using random keys, to hide their true value:

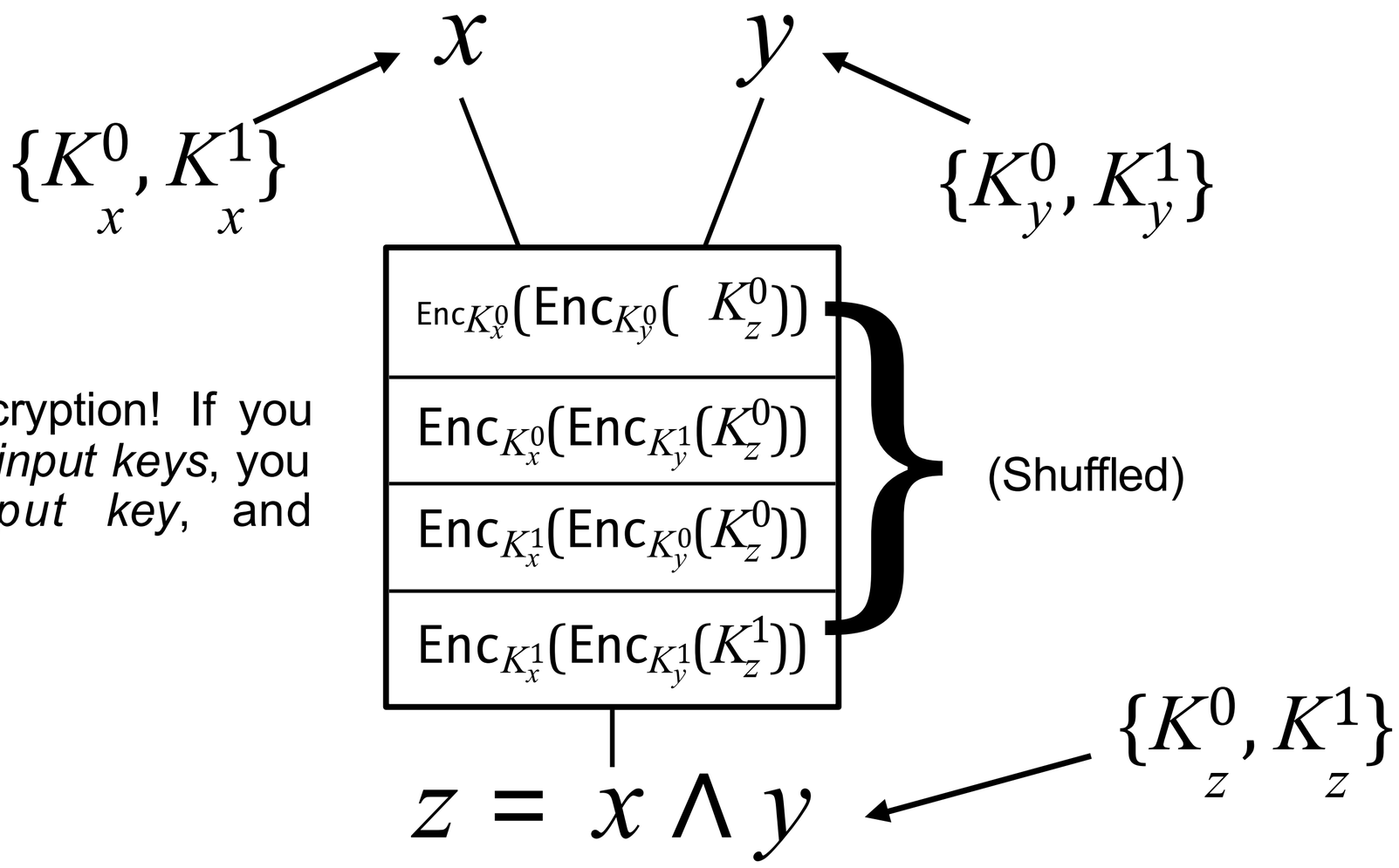


To stay consistent, here we need:

$$K_z^{x \wedge y}$$

Building Block II: Symmetric Encryption

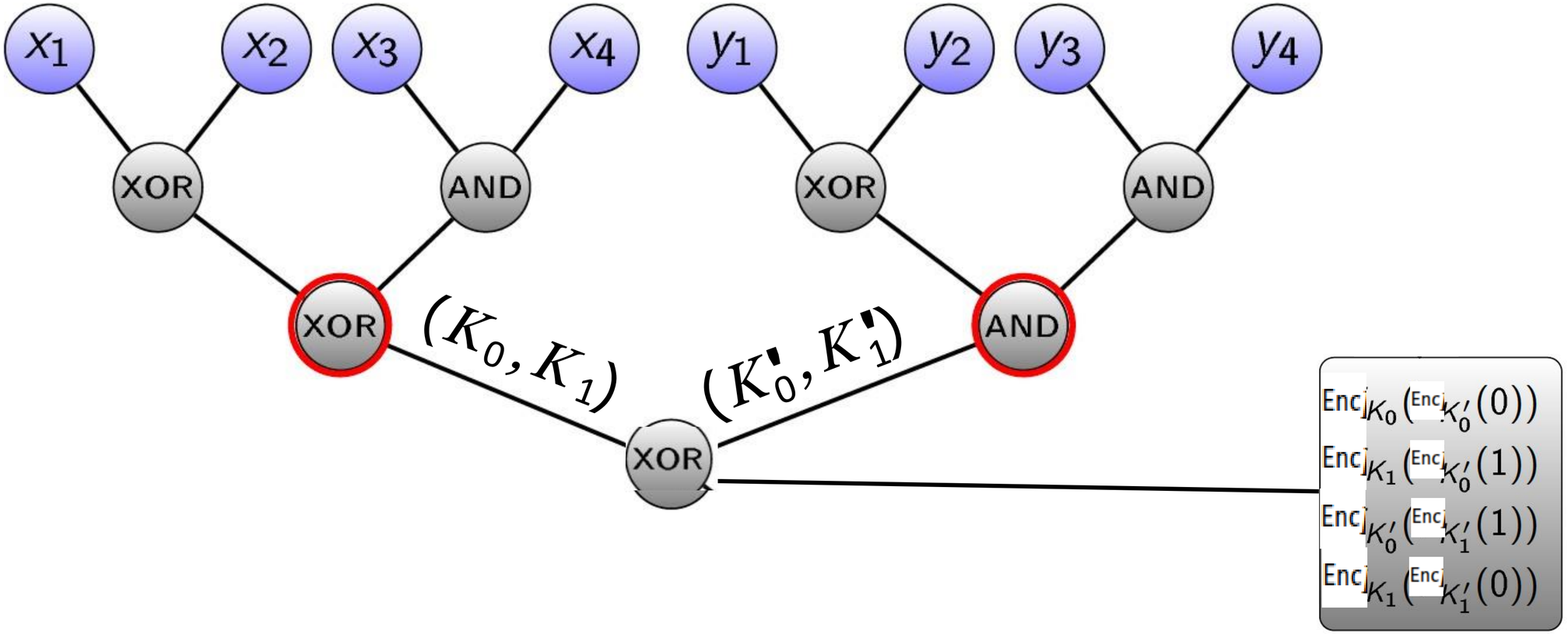
The inputs and outputs are bits, but we will « represent » them using random keys, to hide their true value:



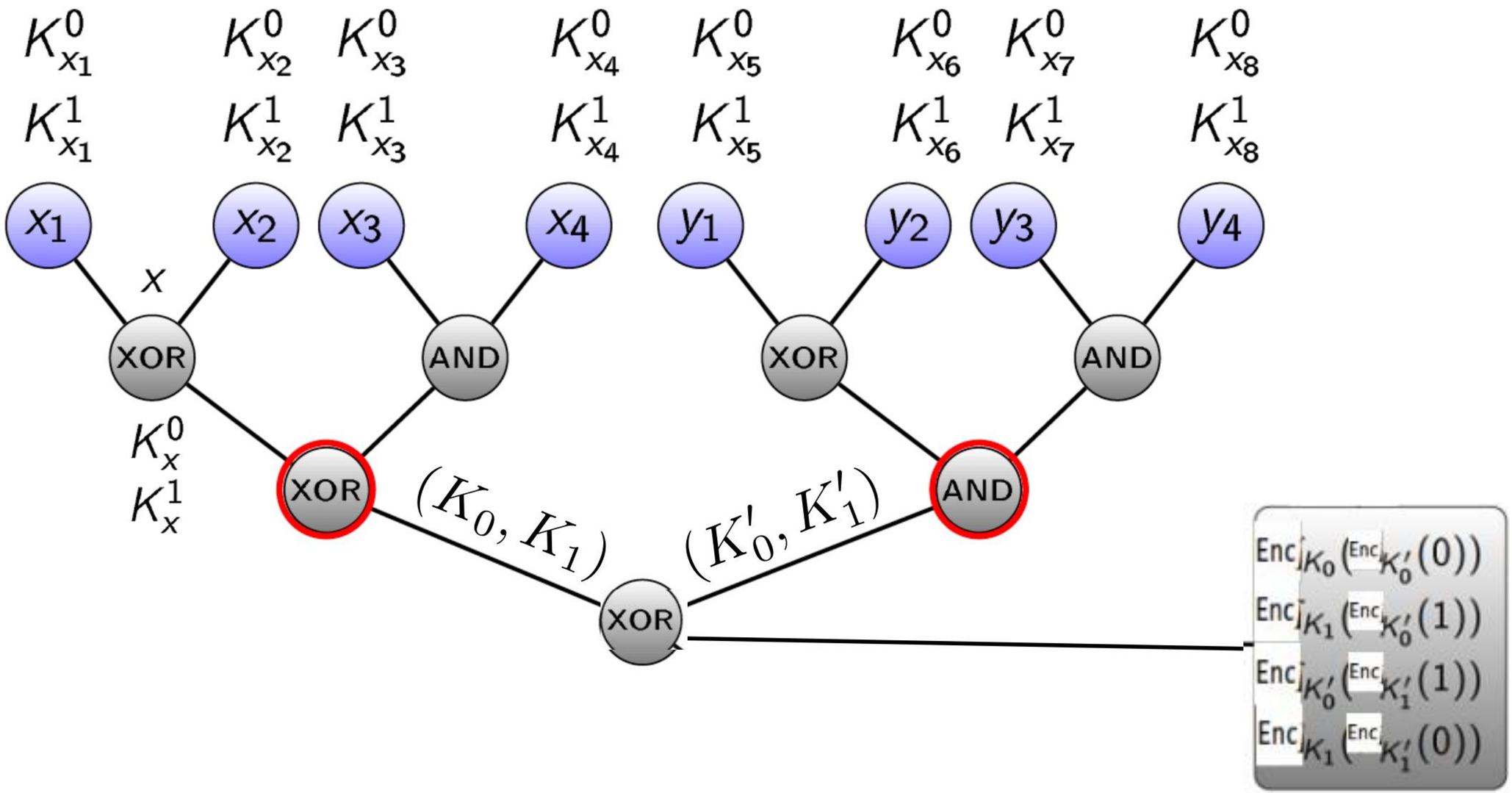
Key idea: double encryption! If you have exactly the right *input keys*, you learn the right *output key*, and nothing more.

Garbled Circuits

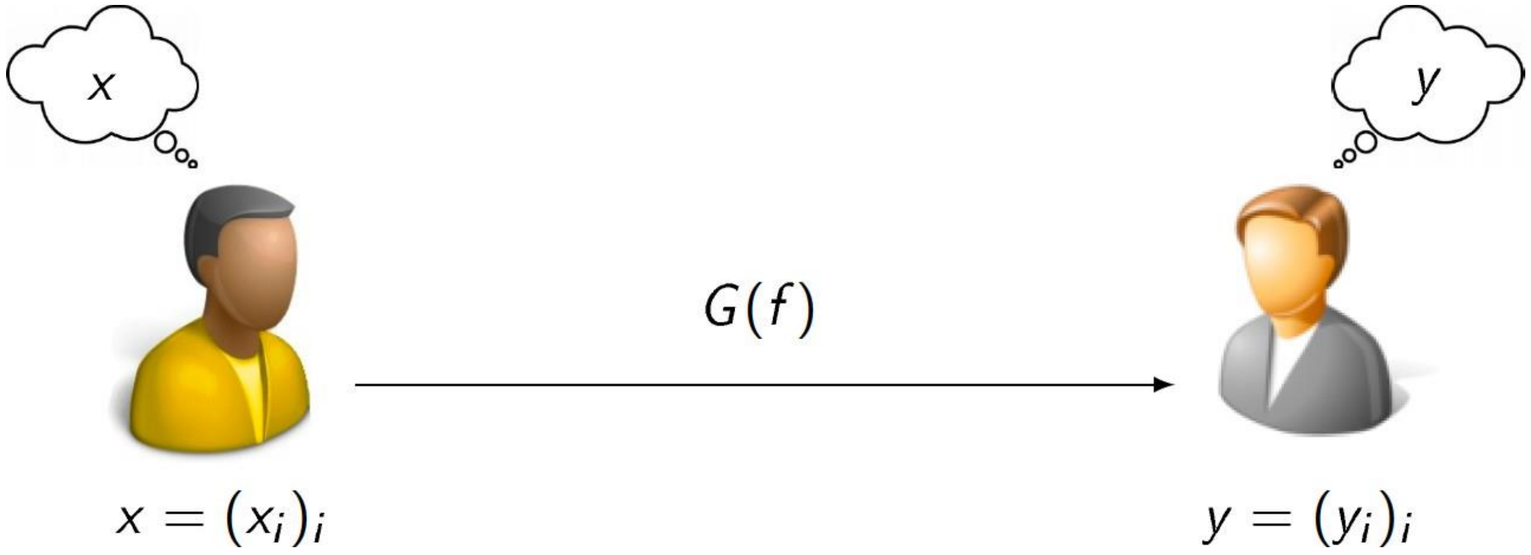
Idea: « encrypting » the gates such that they can only be evaluated given appropriate keys, and while hiding their exact behavior.



Garbled Circuits

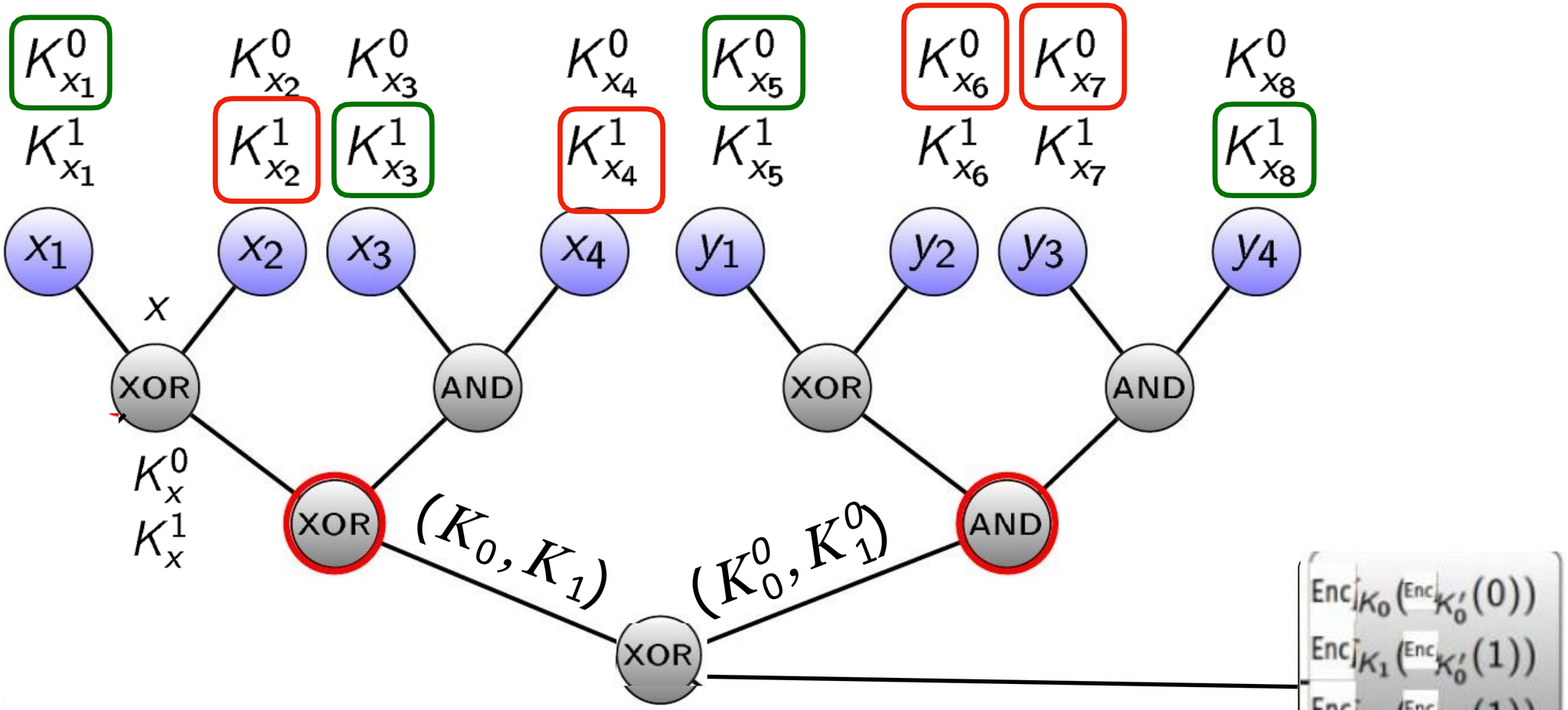


Two-Party Secure Computation for All Functions



It remains to find a way to transmit exactly the appropriate input keys (and nothing more)

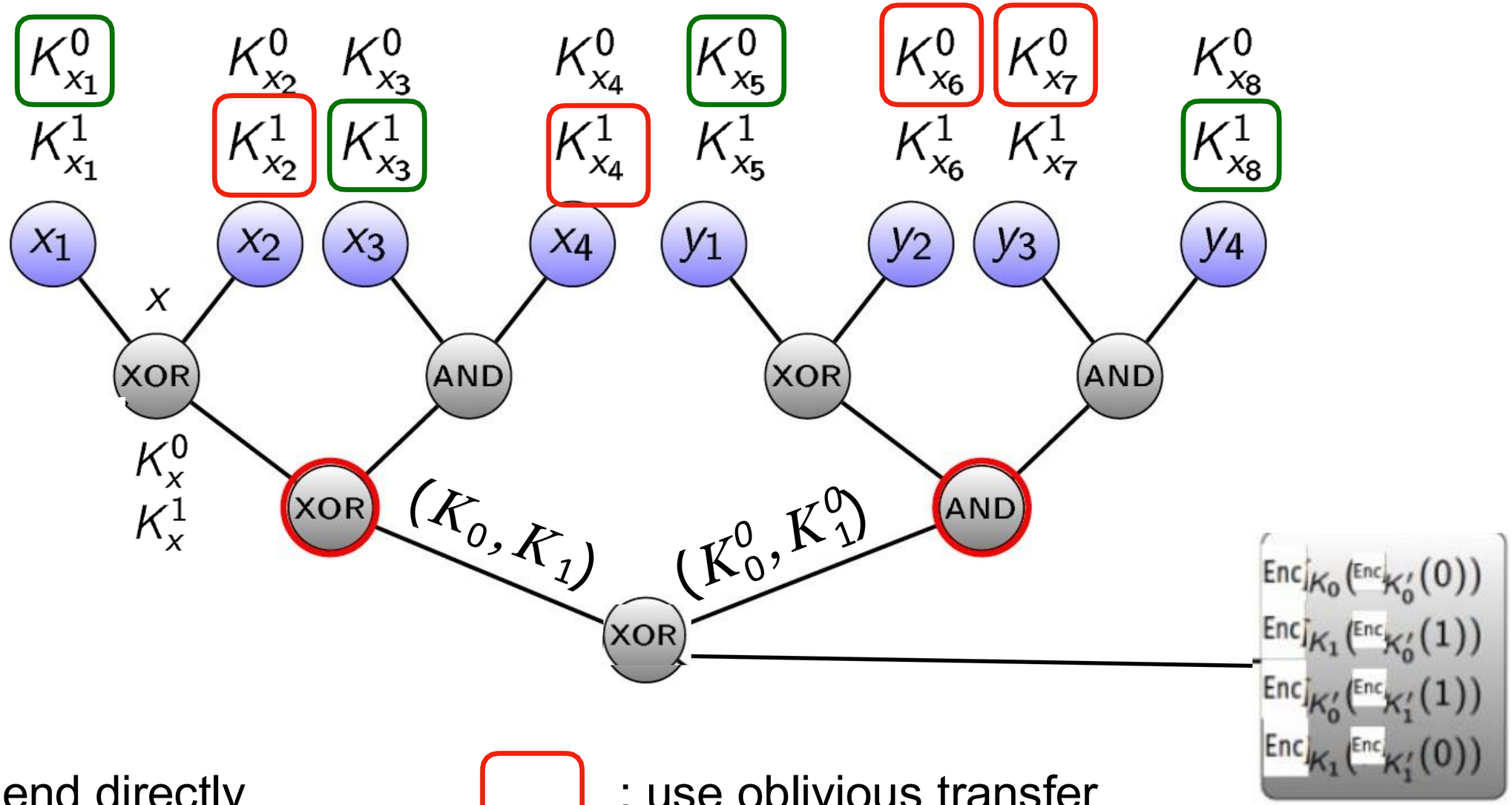
Garbled Circuits



= sender inputs = 0101

= receiver inputs = 1100

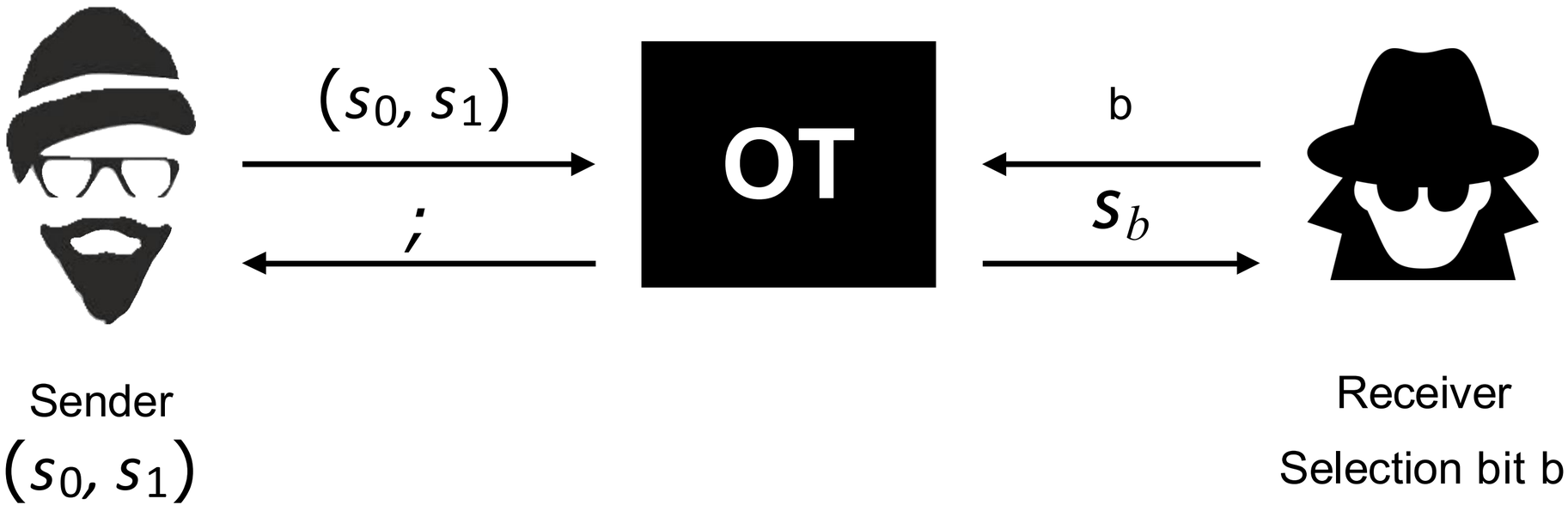
Garbled Circuits



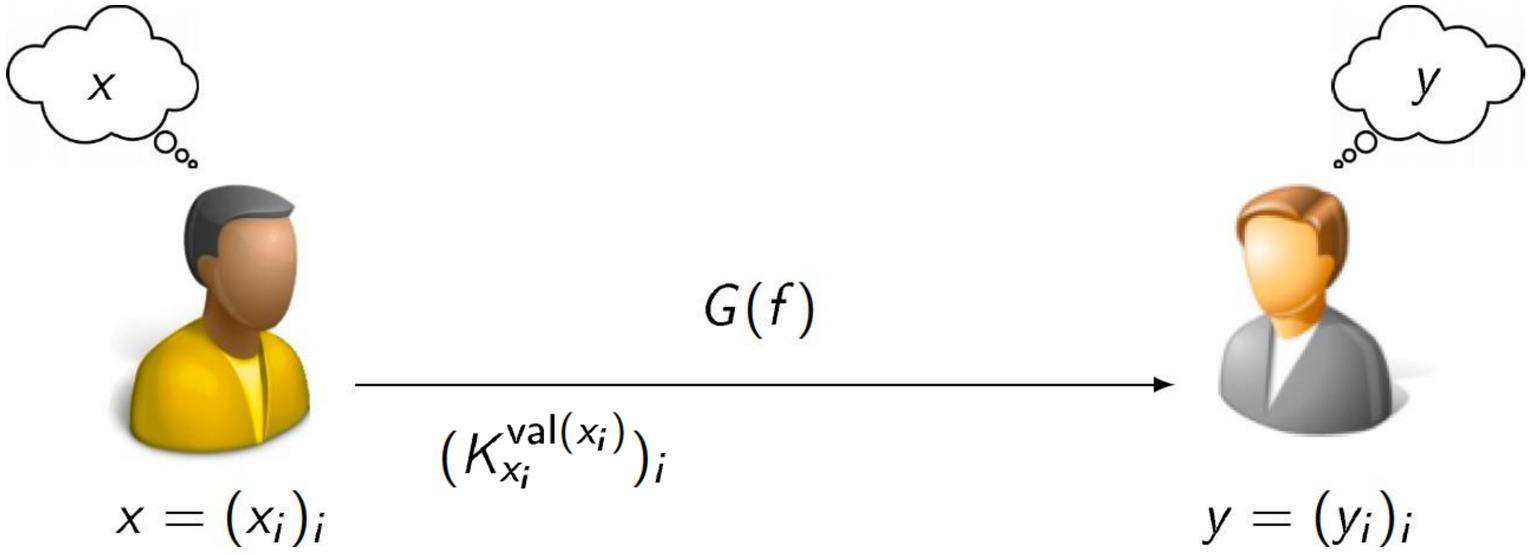
: send directly

: use oblivious transfer

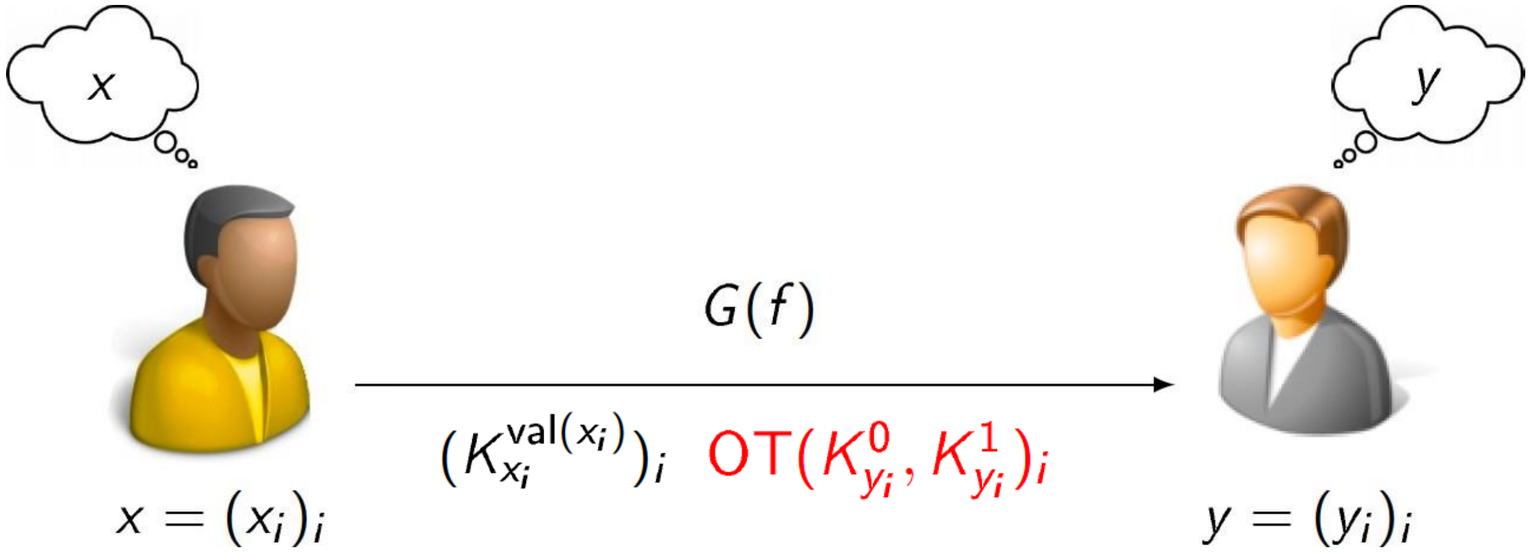
Building Block: Oblivious Transfer



Two-Party Secure Computation for All Functions



Two-Party Secure Computation for All Functions



Simple Example II: Oblivious Transfer

Sender

(s_0, s_1)

Goal :

- The receiver learns s_b
- The sender learns nothing about b
- The receiver learns nothing about s_{1-b}

Receiver

Selection bit b

Simple Example II: Oblivious Transfer

Sender

(s_0, s_1)

Receiver

Selection bit b

Simple Example II: Oblivious Transfer

OT Protocol

Sender(x_0, x_1)		Receiver(b)
		$(pk, sk) \leftarrow \mathsf{Gen}(1^n)$ $pk' \leftarrow \mathsf{Samp}(1^n)$ $pk_b = pk, pk_{1-b} = pk'$
	$\xleftarrow{pk_0, pk_1}$	
$c_0 = \mathsf{Enc}_{pk_0}(x_0), c_1 = \mathsf{Enc}_{pk_1}(x_1)$	$\xrightarrow{c_0, c_1}$	$x_b = \mathsf{Dec}_{sk_b}(c_b)$

Oblivious Transfer - Security Analysis

Security Against the Sender

$\underline{\mathcal{S}(1^n, x_0, x_1)}$:

1. Run $(pk_0, sk_0) \leftarrow \mathsf{Gen}(1^n)$ and $(pk_1, sk_1) \leftarrow \mathsf{Gen}(1^n)$
2. Choose randomness r_0, r_1 for the two encryptions.
3. Output $(pk_0, pk_1, r_0, r_1, x_0, x_1)$.

$\approx$

$\underline{\mathrm{View}^{\pi}_{\mathrm{sender}}(1^n, x_0, x_1)}$:

1. The sender receives $(pk_b, sk_b) \leftarrow \mathsf{Gen}(1^n)$ and $pk_{1-b} \leftarrow \mathsf{Samp}(1^n)$,
2. the randomness r_0, r_1 for the two encryptions.
3. Hence, the sender's view consists of $(pk_0, pk_1, r_0, r_1, x_0, x_1)$.

$\mathsf{Samp}(1^n)$ and $\mathsf{Gen}(1^n)$ are identically distributed.

Oblivious Transfer - Security Analysis

Security Against the Receiver

$\underline{\mathcal{S}(1^n, b, x_b)}$:

1. Choose randomness r_{Gen} and compute $(pk_b, sk_b) \leftarrow \mathsf{Gen}(1^n)$.
2. Run $(pk_{1-b}, sk_{1-b}) \leftarrow \mathsf{Gen}(1^n)$ and compute $r_{\text{Samp}} \leftarrow \mathsf{pkSim}(pk_{1-b})$.
3. Set $c_b \leftarrow \mathsf{Enc}_{pk_b}(x_b)$ and $c_{1-b} \leftarrow \mathsf{Enc}_{pk_{1-b}}(0^n)$.
4. Output $(r_{\text{Gen}}, r_{\text{Samp}}, c_0, c_1, b, x_b)$.

$\approx$

$\underline{\text{View}^{\pi}_{\text{receiver}}(1^n, b)}$:

1. The receiver chooses randomness $r_{\text{Gen}}, r_{\text{Samp}}$ and computes $(pk_b, sk_b) \leftarrow \mathsf{Gen}(1^n; r_{\text{Gen}})$ and $pk_{1-b} \leftarrow \mathsf{Samp}(1^n; r_{\text{Samp}})$.
2. The receiver receives $c_b = \mathsf{Enc}_{pk_b}(x_b), c_{1-b} = \mathsf{Enc}_{pk_{1-b}}(x_{1-b})$.
3. Hence, the receiver's view consists of $(r_{\text{Gen}}, r_{\text{Samp}}, c_0, c_1, b, x_b)$.

pkSim is the algorithm that on input a public key pk outputs the randomness r such that Samp on input r would output pk

Oblivious Transfer - Security Analysis

Security Against the Receiver

Hybrid$(1^n, b)$:

1. Choose randomness r_{Gen} and compute $(pk_b, sk_b) \leftarrow \mathsf{Gen}(1^n; r_{\text{Gen}})$.
2. Compute $pk_{1-b} \leftarrow \mathsf{Gen}(1^n)$ and run pkSim to obtain $r_{\text{Samp}} \leftarrow \mathsf{pkSim}(pk_{1-b})$.
3. Receive ciphertexts $c_b = \mathsf{Enc}_{pk_b}(x_b), c_{1-b} = \mathsf{Enc}_{pk_{1-b}}(x_{1-b})$.
4. Output $(r_{\text{Gen}}, r_{\text{Samp}}, c_0, c_1, b, x_b)$.

By definition of the algorithm pkSim, the distributions $\text{View}^{\pi}_{\text{receiver}}(1^n, b)$ and $\text{Hybrid}(1^n, b)$ are identical. For distributions $\text{Hybrid}(1^n, b)$ and $\mathcal{S}(1^n, b, x_b)$, the difference is that we replaced the encryption of x_{1-b} with that of 0^n. The proof that these two distributions are computationally indistinguishable follows by reduction from the CPA security of the encryption scheme.

Exercise

Write the functionality, a protocol and an analysis for allowing a client to send a message "username = …. password = ….. query =……" to a specific server, out of several servers, receiving back the answer;

the identity of the specific server is known to the client but there are not authenticated channels in the real world;

a corrupted player is semi-honest; the adversary in addition to corrupting a player can read messages exchanged on the communication channel (even without corruption);

the fact that a client queries a specific server is not private;

additionally, consider an adversary that has full control over the communication channel, therefore even updating and blocking messages.

A generic issue: Fairness

Do you recall the leak in the functionality? We need something similar...

The commitment functionality

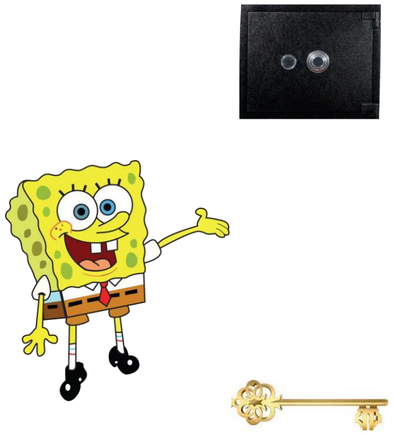
We want a two-step functionality where:

- in a first phase, a user-sender stores a message in a locked safe, and this is notified to a user-receiver
- in a second phase, the user-sender unlocks the safe and the user-receiver gets the message

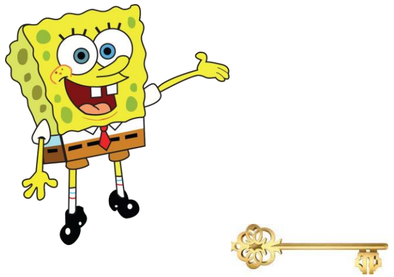
Exercise: realize this functionality assuming the existence of random oracles (i.e., a functionality "consistently" providing random string as output to queries)

Commitment Scheme

Commitment Scheme



Commitment Scheme



Commitment Scheme

Commitment Scheme

Commitment Scheme

A Bit of Formalism: P, NP, Interactive Proofs

Some basics of complexity theory:

- A language L is a set of bitstrings.

- P is the class of all languages which can be *decided* in polynomial time. More formally: a language $\mathcal{L}$ is in P if there exists a polynomial-time Turing machine which, for any n, on input a bitstring $x \in \{0,1\}^n$, runs in time poly(n) and outputs 1 if and only if x belongs to $\mathcal{L}$.

- NP is the class of all languages $\mathcal{L}$ which admit an *efficient* (i.e. polytime) *proof of membership.* More formally: a language $\mathcal{L}$ is in NP if there exists a polynomial-time Turing machine M such that

$$\mathcal{L} = \{x \in \{0,1\}^* : \exists w, |w| = \text{poly}(|x|) \wedge M(x,w) = 1\}.$$

Intuitively, the statements of the form 'x belongs to $\mathcal{L}$' for some NP language $\mathcal{L}$ capture all *efficiently verifiable* statements.

- A language $\mathcal{L}$ is *NP-complete* if $\mathcal{L}$ is in NP, and the existence of a polytime algorithm for $\mathcal{L}$ implies the existence of a poly time algorithm for *any language of NP.*

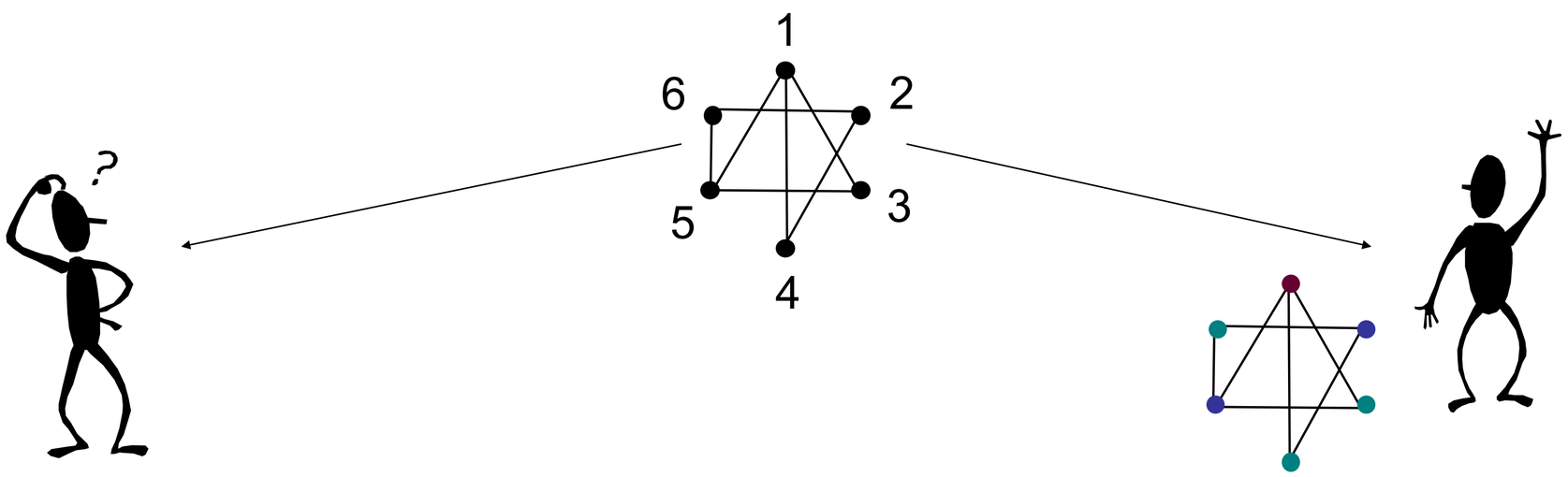
The (ZK) proof of knowledge (PoK) functionality

We want a functionality where: a prover knowing a witness w for x in an NP-language L convinces a verifier that x is in L protecting the privacy of the witness.

This can be realized for all NP languages, an example is 3COL, under minimal hardness assumptions.

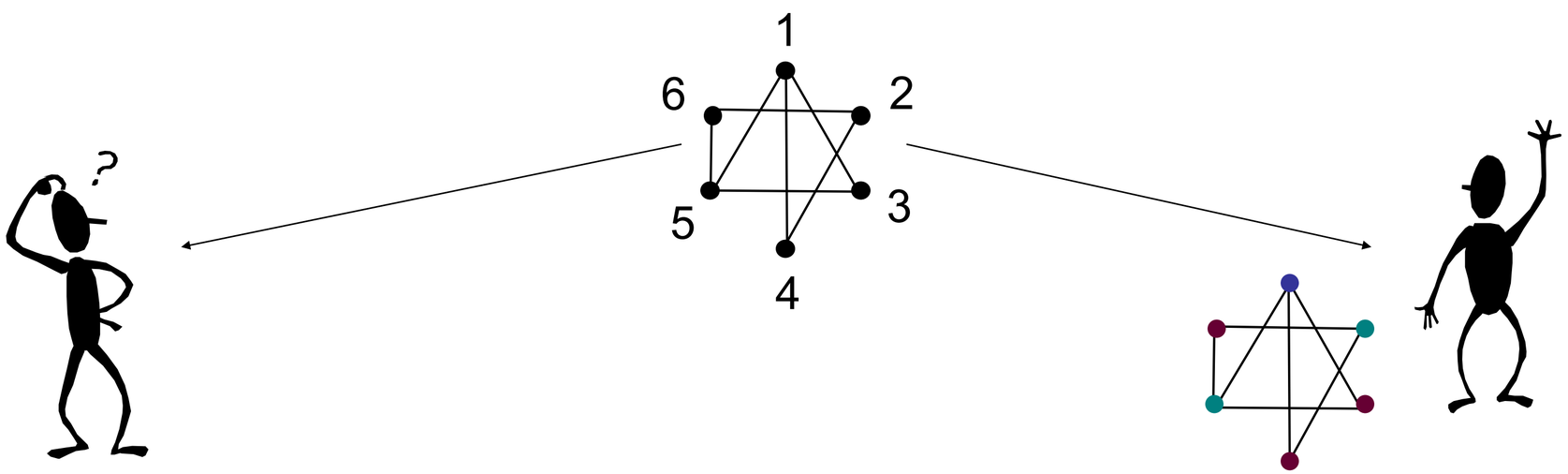
Public-key encryption with "well-formed keys" + ZK proofs realizes the commitment functionality without random oracles.

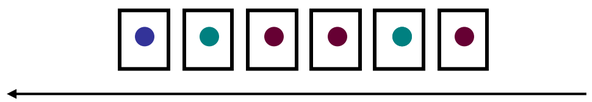
ZK pf for Graph 3-Colorig



1. Randomly permute coloring & send in locked boxes.

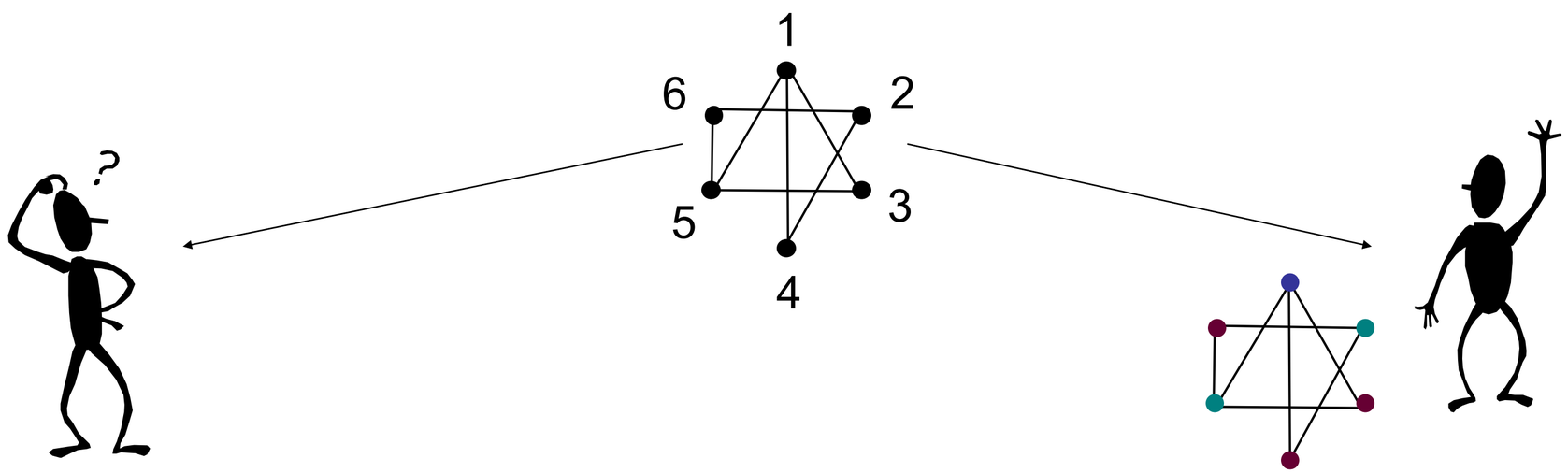
ZK for Graph 3-Coloring





1. Randomly permute coloring & send in locked boxes.

ZK for Graph 3-Coloring



1. Randomly permute coloring & send in locked boxes.

2. Pick random edge.

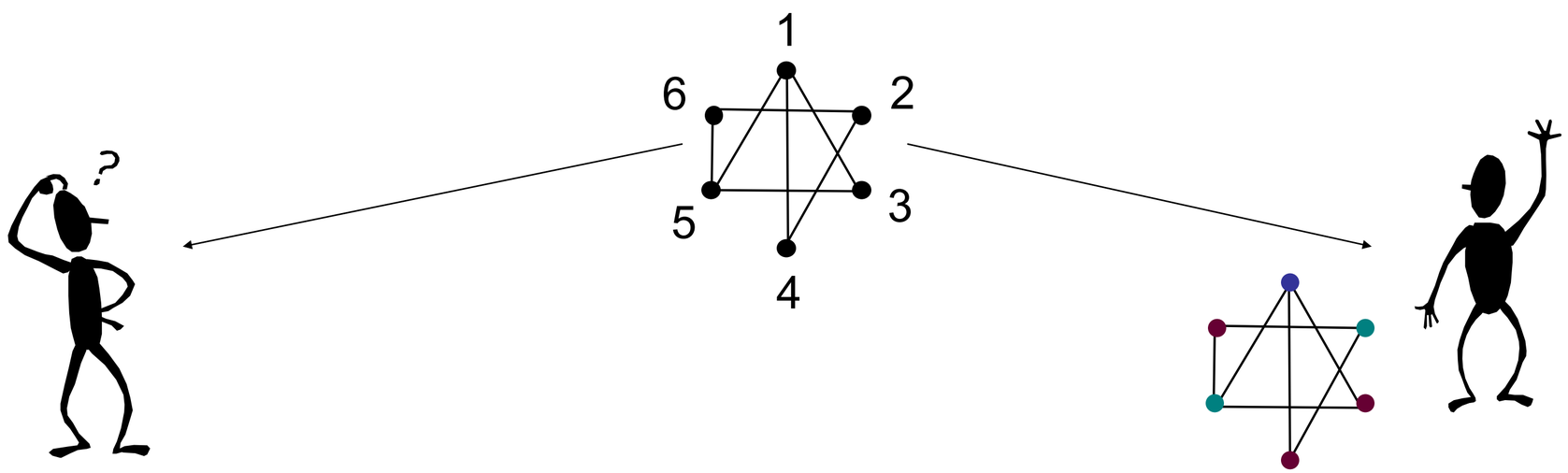
(1,4)

3. Send keys for endpoints.

4. Accept if colors different.

(Perfect) Completeness: graph 3-colorable - V accepts w.p. 1

ZK for Graph 3-Coloring



1. Randomly permute coloring & send in locked boxes.

2. Pick random edge.

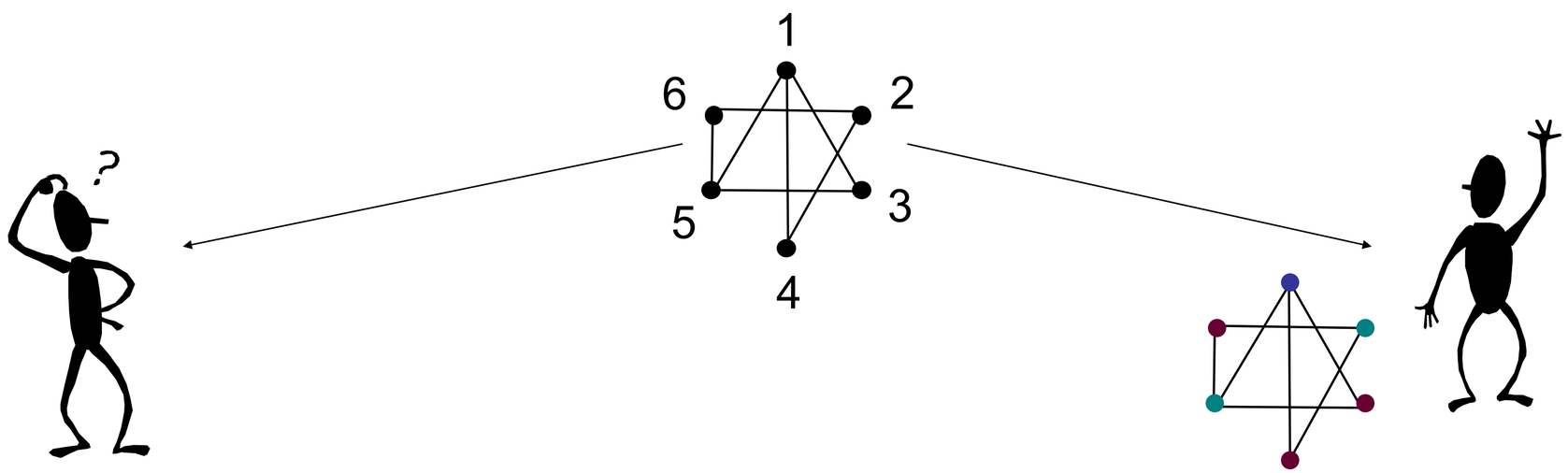
(1,4)

3. Send keys for endpoints.

4. Accept if colors different.

Soundness: graph not 3-colorable V rejects w.p. >=1/(#edges)

ZK for Graph 3-Coloring



1. Randomly permute coloring & send in locked boxes.

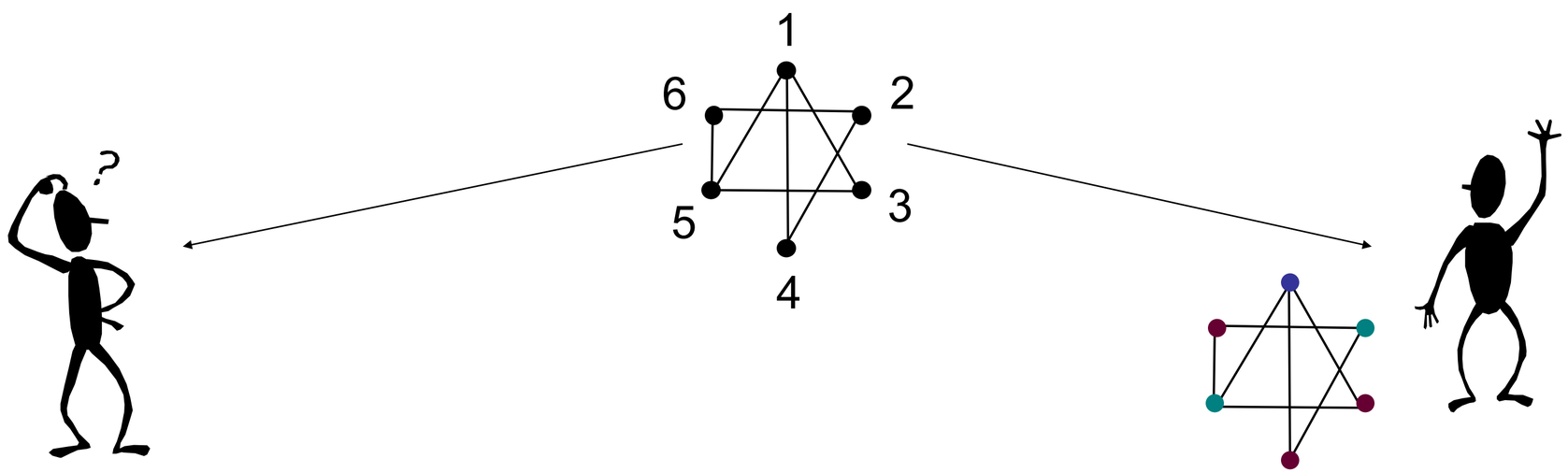
2. Pick random edge.

(1,4)

3. Send keys for endpoints.

4. Accept if colors different.

Zero Knowledge: graph 3-colorable Sim can simulate interaction w/o witness



1. Randomly permute coloring & send in locked boxes.

Com(•)…Com(•)

2. Pick random edge.

(1,4)

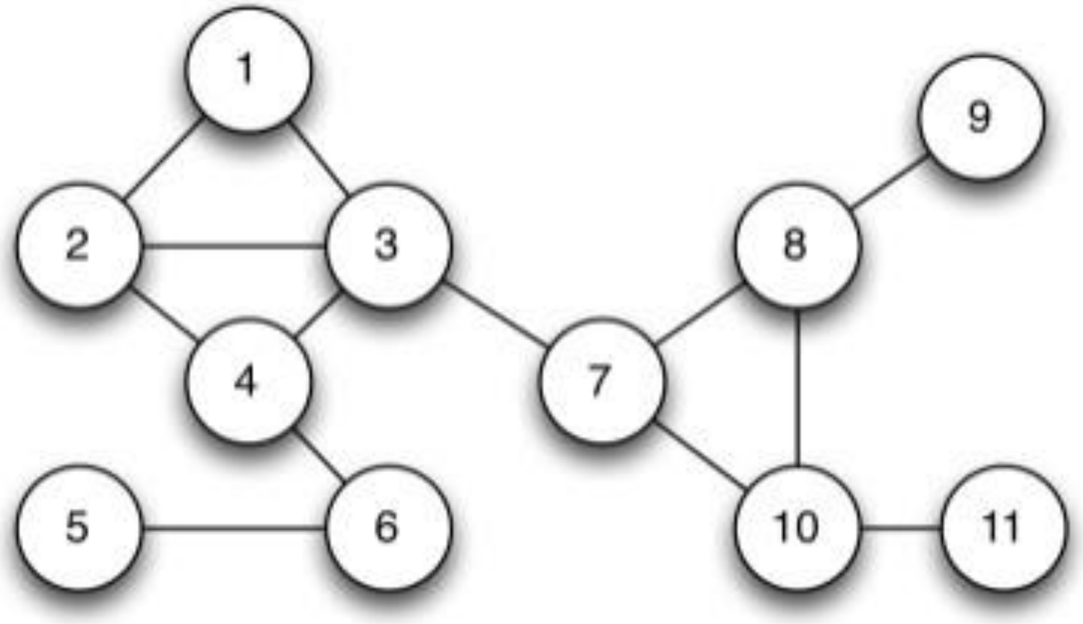
3. Send keys for endpoints.

(•,K_1),(•,K_4)

4. Accept if colors different.

Example: Deploying a Cellular Network

You are a telecom company deploying a new cellular communications network. The graph below is the network structure: nodes are radio towers, edges indicate overlaps. To avoid transmission interferences, each tower can be configured to one of three different frequencies.

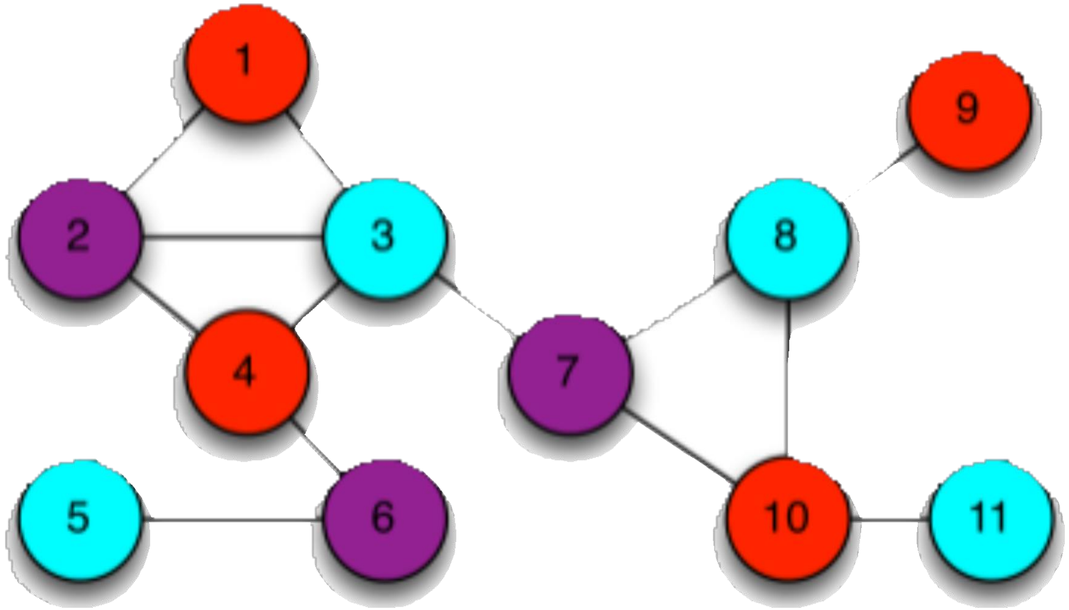


Challenge: assigning a frequency to each tower such that no interference can occur. Some of you might have recognized an instance of the *3-coloring* graph problem.

Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

Example: Deploying a Cellular Network

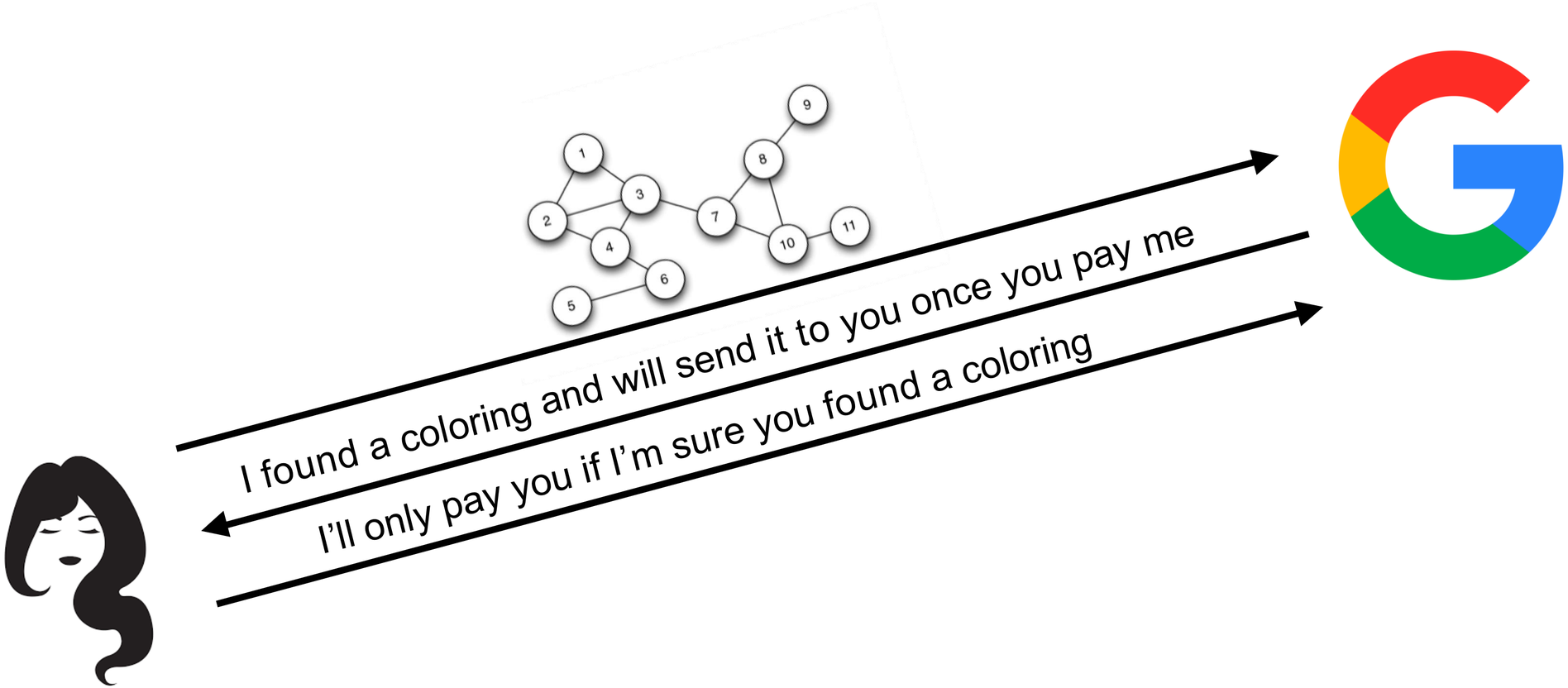
You are a telecom company deploying a new cellular communications network. The graph below is the network structure: nodes are radio towers, edges indicate overlaps. To avoid transmission interferences, each tower can be configured to one of three different frequencies.



Challenge: assigning a frequency to each tower such that no interference can occur. Some of you might have recognized an instance of the *3-coloring* graph problem.

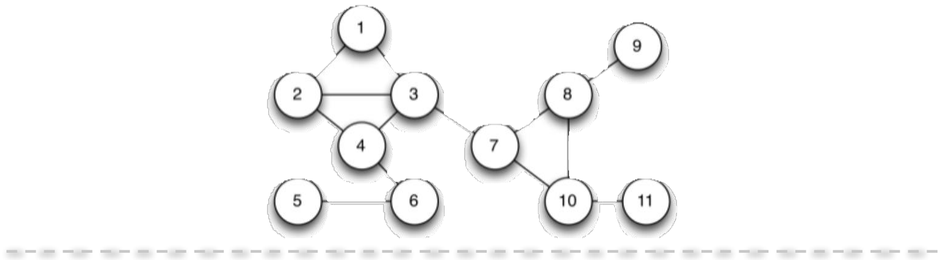
Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

Example: Deploying a Cellular Network



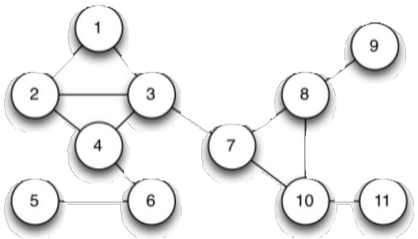
Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

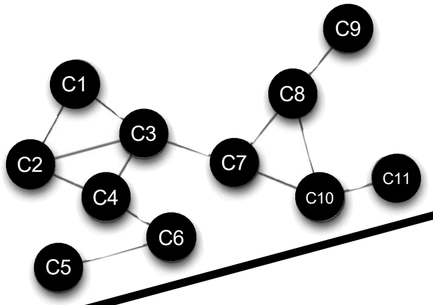
A Zero-Knowledge Protocol



Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol

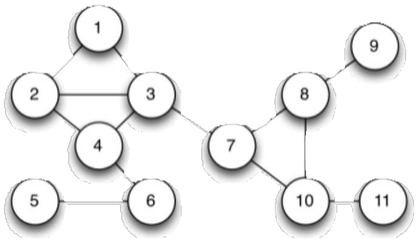


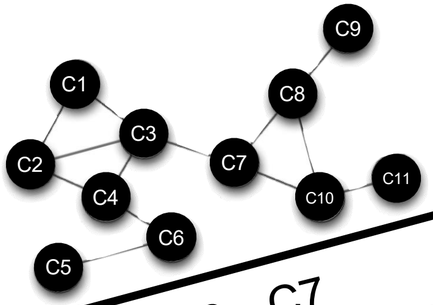


- Pick an association: colors {red,bue,purple} <-> letters {A,B,C}
- Find a coloring and commit to the letters for each node.

Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol





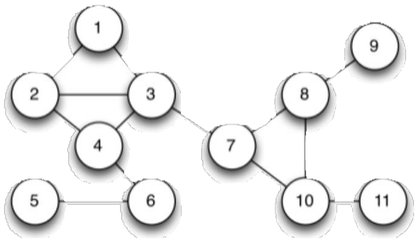
- Pick an edge and challenge Google to open the nodes.

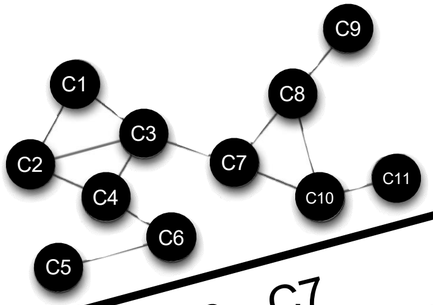
Open C3 - C7

- Pick an association: colors {red,bue,purple} <-> letters {A,B,C}
- Find a coloring and commit to the letters for each node.

Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol





- Pick an edge and challenge Google to open the nodes.

Open C3 - C7

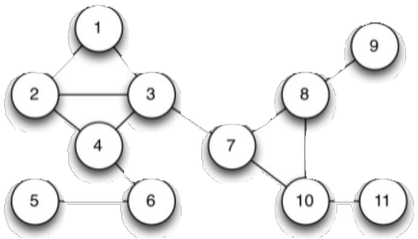
('A', d3), ('B', d7)

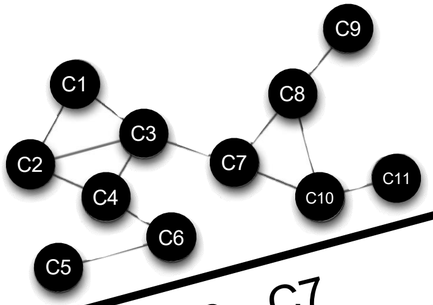
- Pick an association: colors {red,bue,purple} <-> letters {A,B,C}
- Find a coloring and commit to the letters for each node.

- Send the opening to Alice's query.

Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol





- Pick an edge and challenge Google to open the nodes.

Open C3 - C7

('A', d3), ('B', d7)

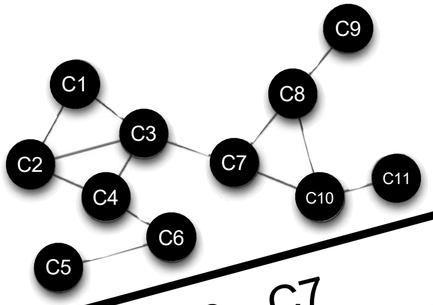
- Pick an association: colors {red,bue,purple} <-> letters {A,B,C}
- Find a coloring and commit to the letters for each node.
- Send the opening to Alice's query.

- Accept if Verify('letter3',d3) = Verify('letter7',d7) = 1 *and* letter3 =/= letter7; otherwise, reject.

Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol

Correctness: check it.



- Pick an edge and challenge Google to open the nodes.

Open C3 - C7

('A', d3), ('B', d7)

- Pick an association: colors {red,bue,purple} <-> letters {A,B,C}
- Find a coloring and commit to the letters for each node.

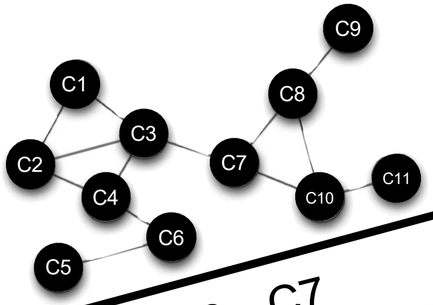
- Send the opening to Alice's query.

- Accept if Verify('letter3',d3) = Verify('letter7',d7) = 1 *and* letter3 =/= letter7; otherwise, reject.

Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol

Zero-Knowledge



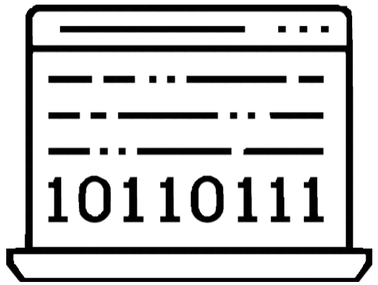
- Pick an edge and challenge Google to open the nodes.

Open C3 - C7

('A', d3), ('B', d7)

- Pick a random edge, commit to two random letter in {A,B,C} for this edge, and 0 for all other nodes.

- If Alice's code return this edge, open it, otherwise, restart.



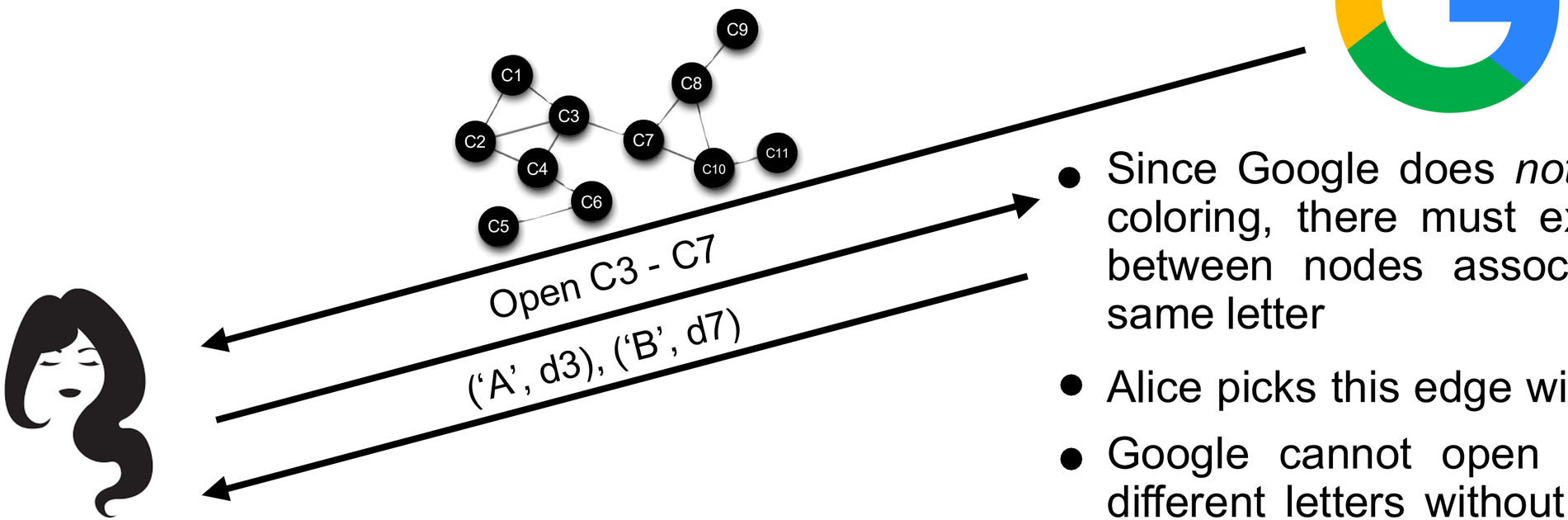
Convince yourself that this produces a transcript indistinguishable from a honest transcript

Example taken from a great blog post by Matthew Green:

https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol

Soundness: 1/|E|



- Since Google does *not* know a 3 - coloring, there must exist an edge between nodes associated to the same letter
- Alice picks this edge with prob 1/|E|
- Google cannot open this edge to different letters without breaking the binding property of the commitment.

Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol

Repeat all n*|E| times

$$\left(1 - \frac{1}{|E|}\right)^{n \cdot |E|} = e^{-n}$$

- Since Google does *not* know a 3 - coloring, there must exist an edge between nodes associated to the same letter
- Alice picks this edge with prob 1/|E|
- Google cannot open this edge to different letters without breaking the binding property of the commitment.

Example taken from a great blog post by Matthew Green:
https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/

A Zero-Knowledge Protocol for *any* Statement

- We just gave a zero-knowledge proof system for membership to the 3-colorable language

- 3-coloring is NP-complete: *any* problem in NP can be transformed (in polynomial time) into an instance of the 3-coloring problem

- This gives a zero-knowledge proof for any problem in NP: take the target NP problem, transform it into an instance of 3-coloring (the word is mapped to a graph and the NP-witness is mapped to a 3-coloring of the graph), and use the zero-knowledge proof to show knowledge of the witness.

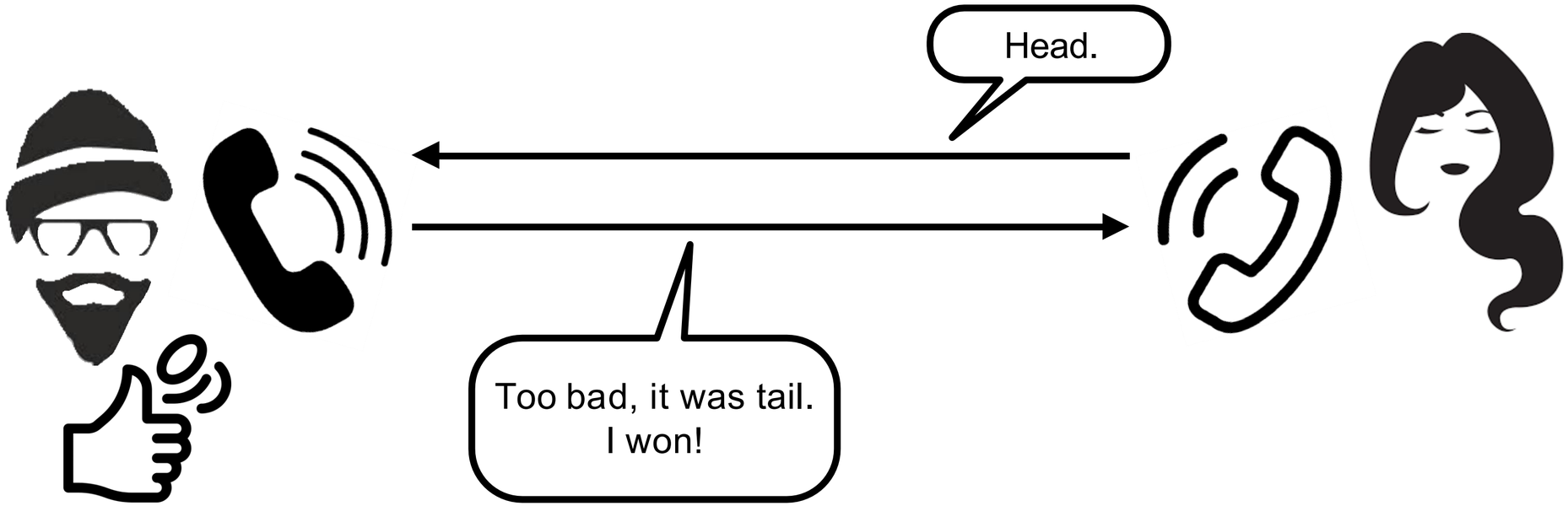
Getting a zero-knowledge proof *of knowledge*

- We only gave a zero-knowledge proof of *membership.* To get a zero-knowledge proof of knowledge, we need a property stronger than soundness: *extractability.*

- There are standard techniques to achieve this stronger property. Example: use an *extractable* commitment (for example, a public key encryption scheme: the secret key is the *extraction trapdoor*).

Commitment Scheme: Coin-Flipping Over the Phone

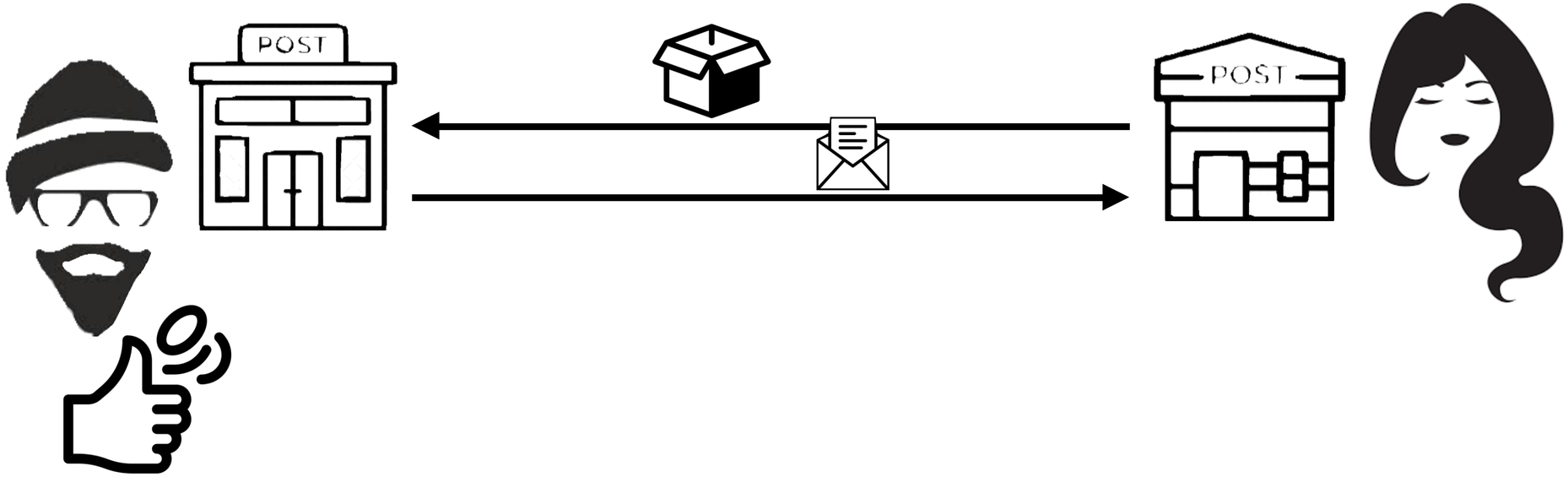
Alice and Bob want to flip a coin over the phone:



How can they prevent cheating behaviors?

Commitment Scheme: Coin-Flipping Over the Phone

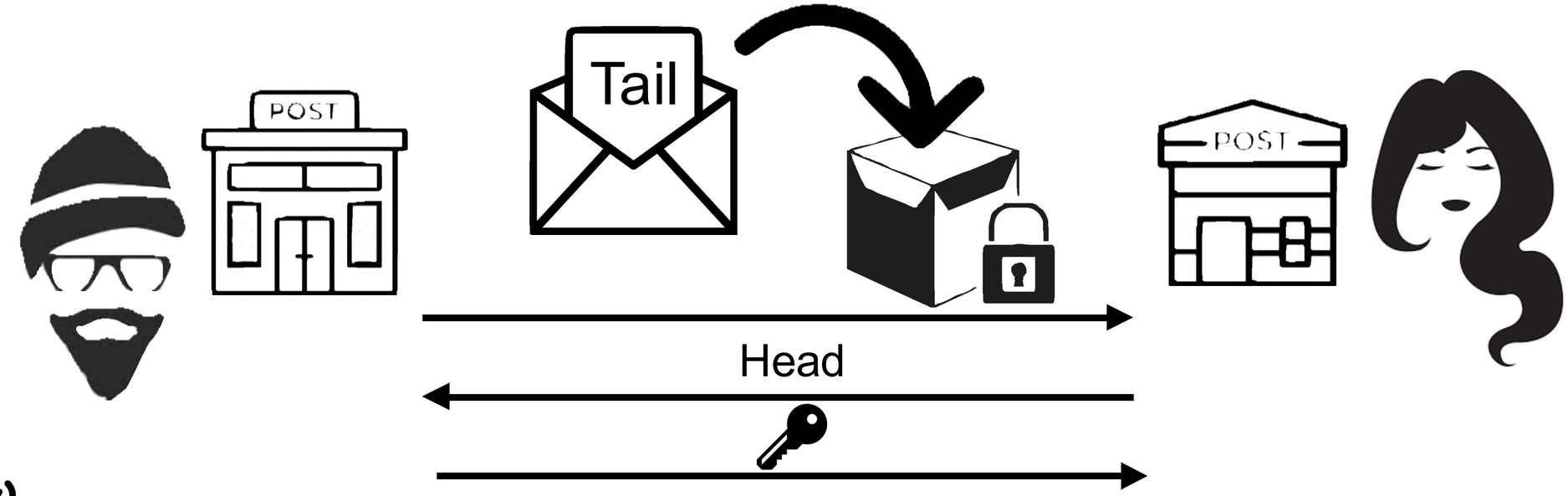
Let’s first see how they could proceed using a post office



Any idea?

Solution

Let’s first see how they could proceed using a post office



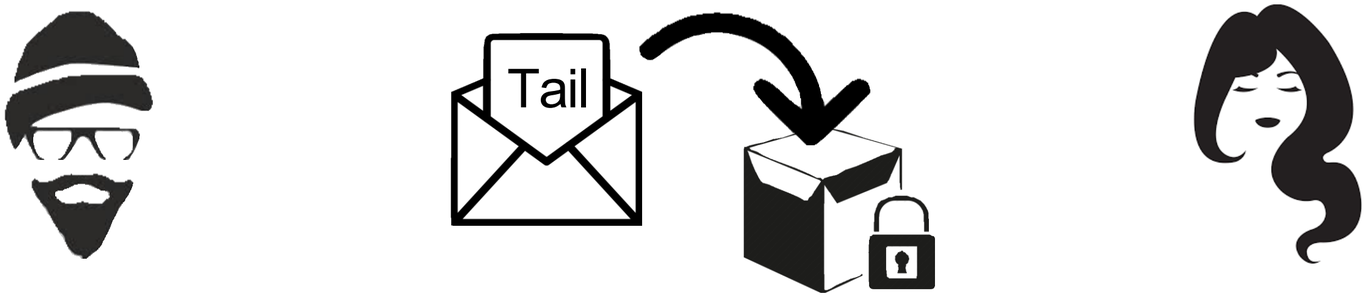
1 Bob flips a coin, writes the result on a letter, which he puts into a locked box.

2 Alice picks head or tail and tells it to Bob.

3 Bob announces the result and sends the key to the box, proving that the result of the coin flip was fixed before Alice made her choice.

Solution - Core Component

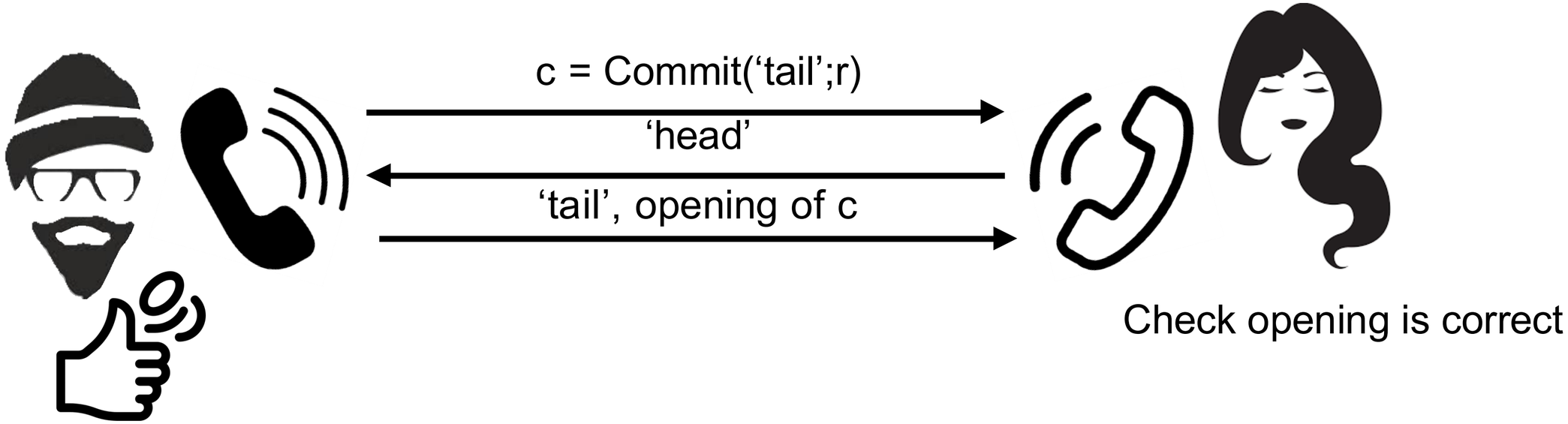
The core object used to solve the problem is a box with a message inside. Looking at the solution, there are two properties which we wanted from this object:
- When she receives a box with a message inside, Alice cannot see this message
- When Bob reveals the content of the box, he cannot lie about what was actually inside



A **commitment scheme** is a cryptographic primitive securely implementing the commitment functionality. For simplicity we will use the keywords Commit and Open to denote interactive algorithms allowing a sender to commit and to open the commitment

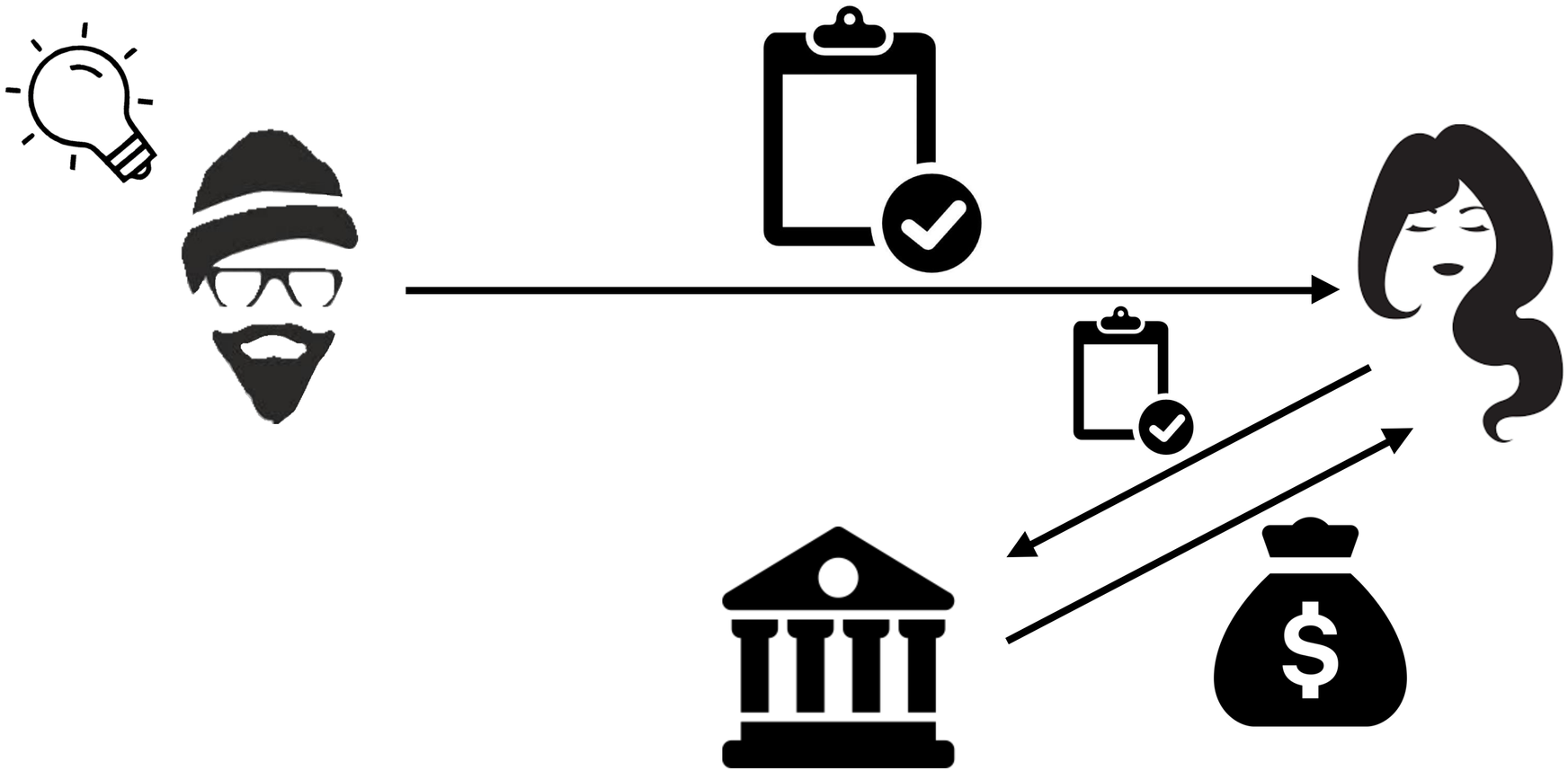
Commitment Scheme: Coin-Flipping Over the Phone

We can finally provide a full solution to our problem:



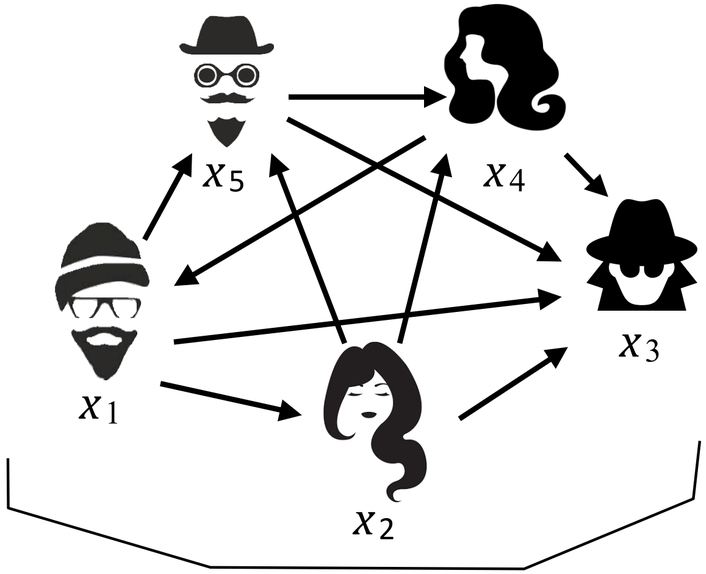
Second Tool: Zero-Knowledge Proofs

Scenario: you just solved a millenium problem, can you prove it to your friends?

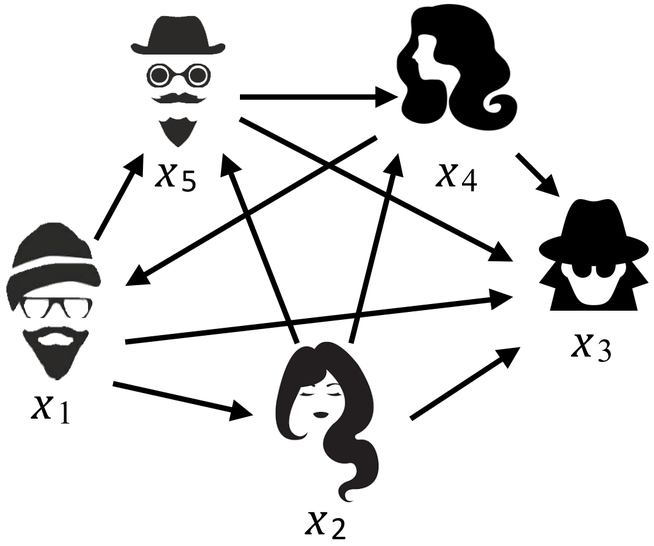


Wrapping Up: Secure Computation against Malicious Adversaries

Honest-But Curious Protocol



Malicious Protocol



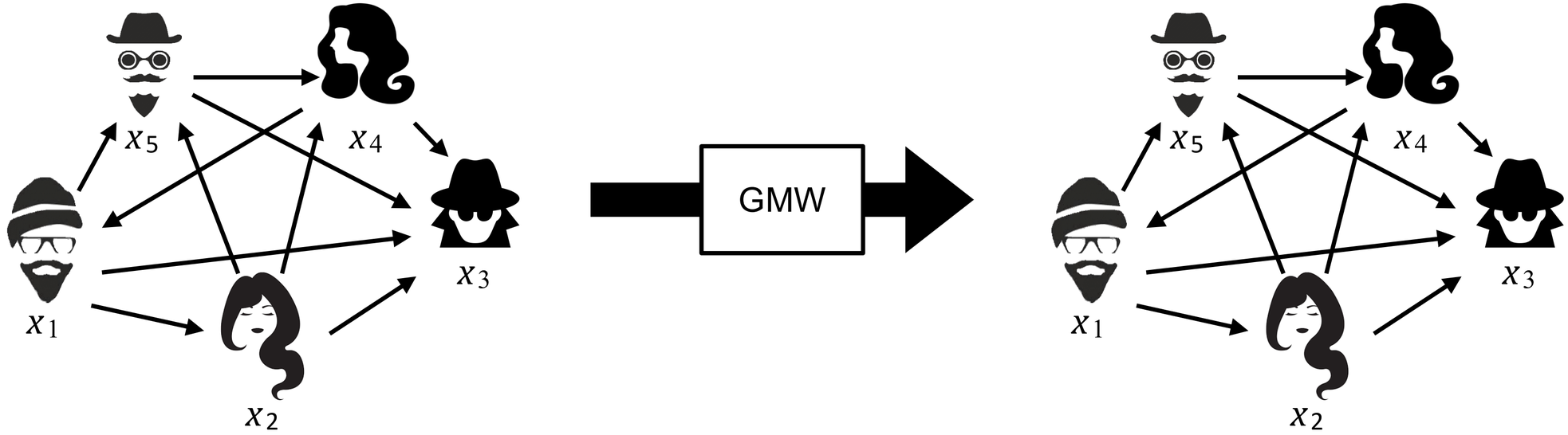
- The honest-but-curious protocol is in the **broadcast model**: each message is sent to everyone.

- Formalize the protocol as being a *function* NextMessage: NextMessage takes as input
  - The input of the current player
  - The random tape of the current player
  - The current transcript (i.e., the list of all message received during the protocol so far)

  and outputs the next message sent by the current player to all players.

Wrapping Up: Secure Computation against Malicious Adversaries

Honest-But Curious Protocol	Malicious Protocol

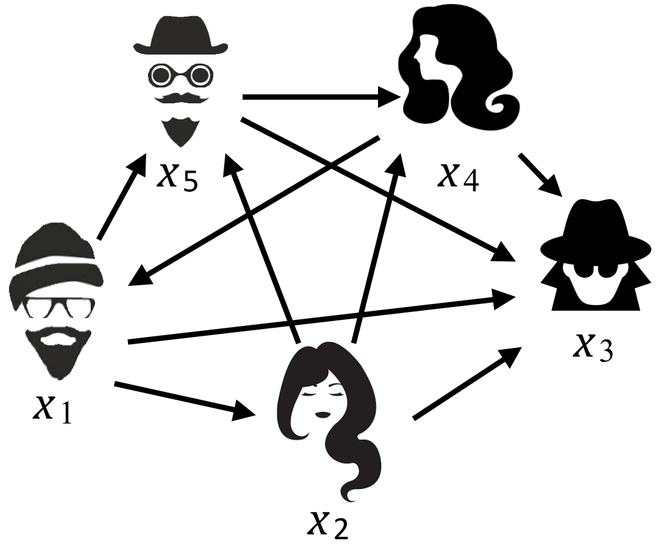


- Start from the honest-but-curious protocol. Let all parties use a **commitment** to commit to their input and their random tape before starting the protocol.

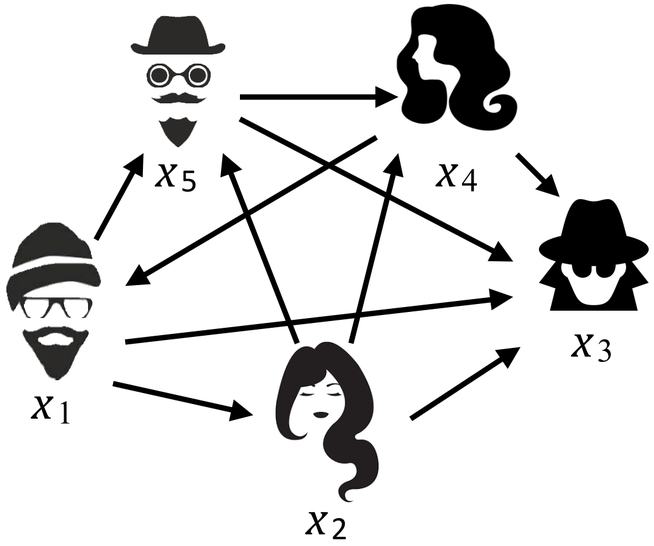
- Each time a player sends a message in the honest-but-curious protocol, he additionally performs a **zero-knowledge proof of knowledge** with *all other players in the protocol*, to prove the following statement: 'I know an input x and a random tape r such that (1) the commitments sent in the first flow are valid commitments to x and r, and (2) the message I just send is equal to NextMessage(x, r, transcript)

Wrapping Up: Secure Computation against Malicious Adversaries

Honest-But Curious Protocol



Malicious Protocol

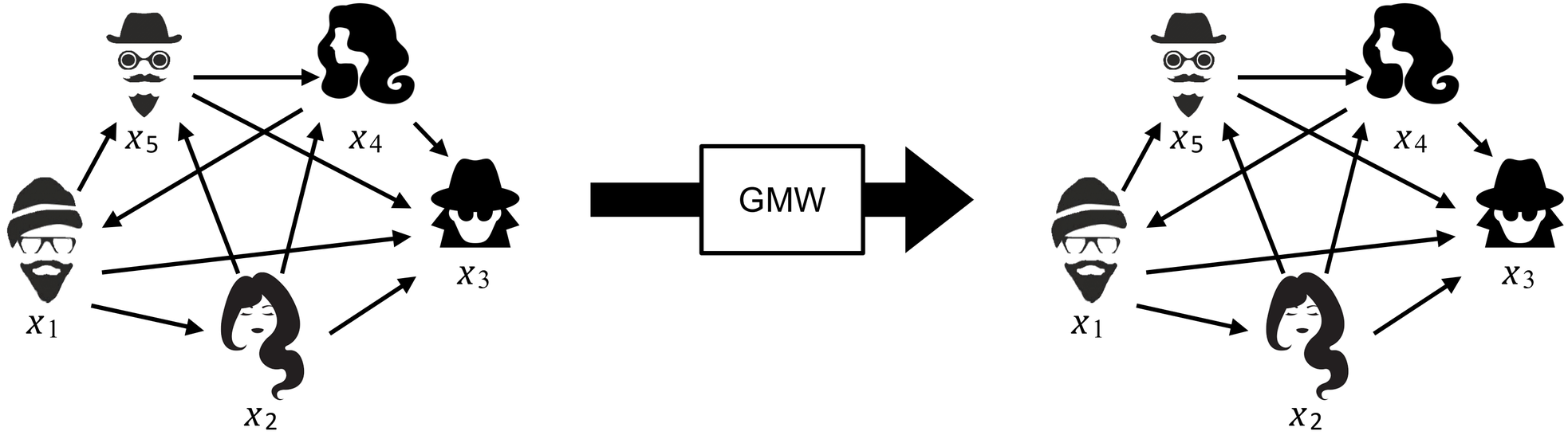


- Start from the honest-but-curious protocol. Let all parties use a **commitment** to commit to their input and their random tape before starting the protocol.

- Each time a player sends a message in the honest-but-curious protocol, he additionally performs a **zero-knowledge proof of knowledge** with *all other players in the protocol*, to prove the following statement: 'I know an input x and a random tape r such that (1) the commitments sent in the first flow are valid commitments to x and r, and (2) the message I just send is equal to NextMessage(x, r, transcript) **=> this is an NP statement with witness (x, r, randomness_commit)!**

Wrapping Up: Secure Computation against Malicious Adversaries

Honest-But Curious Protocol	Malicious Protocol

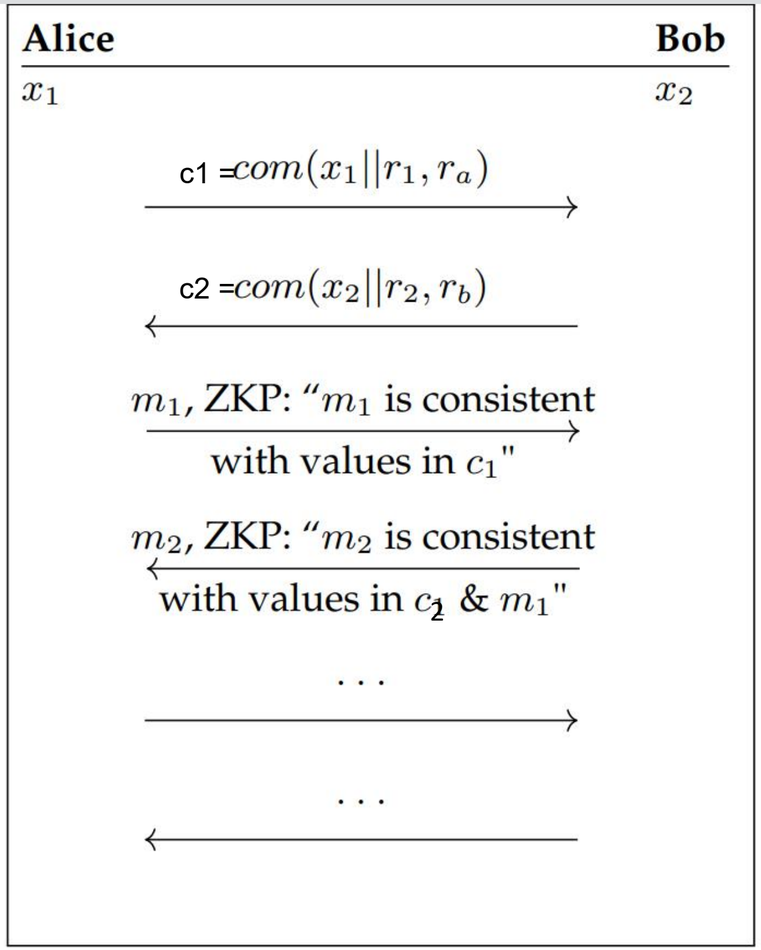


- Security (intuition): the zero-knowledge proofs do not harm the semi-honest security of the underlying protocol (proof: replace the parties by the ZK simulator). If the parties follow the protocol, we know that it securely emulates the functionality. But if any party ever deviates from the protocol, then by soundness of the ZK proof, he gets ‘caught’ by everyone, and the protocol terminates (it is equivalent to leaving the protocol, which is always an option).

From Semi-Honest Security to Malicious Security

What is wrong here?

The randomnesses r1 and r2 should be established with a coin flipping protocol



GMW Compiler

GMW Compiler: with proof sketch

https://www.cs.umd.edu/~jkatz/gradcrypto2/f13/lecture20.pdf

Exercise: let's apply the GWM compiler to the semi-honest OT protocol